



Hochschule
Bonn-Rhein-Sieg
University of Applied Sciences



R&D Project

Recognizing Textual Entailment : Analysis of Deep Learning Models

Ramit Sharma

Submitted to Hochschule Bonn-Rhein-Sieg,
Department of Computer Science
in partial fulfillment of the requirements for the degree
of Master of Science in Autonomous Systems

Supervised by

Prof. Dr. Paul G. Plöger
M.Sc. Deebul Nair

November 2019

I, the undersigned below, declare that this work has not previously been submitted to this or any other university and that it is, unless otherwise stated, entirely my own work.

Date

Ramit Sharma

Abstract

Entailment is the directional relationship between a given pair of sentences, such that if the first sentence referred to as text is true, then second sentence referred as hypothesis must also be true. Hence, Recognizing Textual Entailment (RTE) is the task to classify the directional relationship between text and hypothesis into three different categories, namely entailment, contradiction and neutral. Over the years researchers have tried to solve the problem of RTE in a number of ways, but the major success has been obtained by the application of deep learning. Hence, in this project, a survey of deep learning based models used in the field of RTE is done, and comparative evaluation of four deep learning based models namely decomposable attention, asymmetric attention, LSTM and GRU is carried out with respect to the performance, time for training and number of trainable parameters. The models were tested on SNLI corpus and it was found that decomposable attention based model achieved an accuracy of 86 percent where as LSTM, GRU and asymmetric attention models achieved an accuracy of 75, 74 and 81 percent respectively, and hence decomposable attention model outperformed LSTM, GRU and asymmetric attention model. It was observed that decomposable attention model has fewer trainable parameters and has significantly reduced training time compared to LSTM and GRU.

Acknowledgements

First of all, I would like to thank my parents for supporting me. I want to express my gratitude to Prof. Dr. Paul G. Plöger, for guiding me in this project. I would also like to thank MSc. Deebul Nair for constant support and the insights which he provided throughout this project. I would also like to thank Msc. Santosh Thoduka and MSc. Lakshadeep Naik for supporting me and reviewing the report.

Contents

1	Introduction	1
2	State of the Art	5
2.1	Non-Deep learning approaches	5
2.1.1	Tree Edit Distance Algorithms	5
2.1.2	Semantic Approach	6
2.1.3	Lexical Similarity Approach	7
2.1.4	Shallow and Deep NLP Methods	8
2.2	Deep Learning Approaches	8
2.2.1	Long Short-Term Memory-Networks(LSTM)	8
2.2.2	Reinforcement Learning	11
2.2.3	Reasoning with Entailment	12
2.2.4	Tree-Based Convolution and Heuristic Matching	12
2.2.5	BERT	13
2.3	Directional Self Attention Network	17
2.4	Decomposable Attention Model	18
2.4.1	Attend	19
2.4.2	Compare	19
2.4.3	Aggregate	20
2.5	Asymmetric Attention Model	20
2.6	Discussion About the Models	22
3	Methodology	23
3.1	Neural Network	23
3.1.1	Neuron	23
3.1.2	Single-Layer Feedforward Networks	25

3.1.3	Multi-Layer Feedforward Networks	25
3.1.4	Recurrent Neural Network	26
3.1.5	LSTM	27
3.1.6	GRU	30
3.1.7	Dropout	32
3.1.8	Word Embedding	32
3.2	Corpus	35
3.2.1	SNLI	36
3.2.2	Multi-NLI	38
3.2.3	SCITAIL	40
3.3	Implementation	42
3.3.1	Decomposable Attention Model	42
3.3.2	Asymmetric attention model	47
3.3.3	LSTM	48
3.3.4	GRU	49
4	Evaluation	51
4.1	Activation function	52
4.2	Evaluation of models	53
4.3	Datasets	55
4.4	Dropout	57
4.5	Analysis of Sentences	58
4.5.1	active-passive sentences	60
5	Conclusions	63
Appendix A Deployment of Deep learning model in development server		65
Bibliography		67

List of Figures

2.1	BERT input representation [16]	14
3.1	Impact of bias on the net input [25]	24
3.2	Basic Artificial Neuron [13]	24
3.3	Basic Artificial Neuron [28]	25
3.4	Multi-Layer Feedforward Networks [50]	26
3.5	Loop in Recurrent Neural Network [42]	26
3.6	unrolled recurrent neural network [42]	27
3.7	LSTM network architecture [42]	28
3.8	Cell state [42]	28
3.9	Input gate [42]	29
3.10	cell state [42]	29
3.11	output cell state [42]	30
3.12	GRU architecture [42]	31
3.13	Dropout [27]	32
3.14	first data of SNLI dataset [52] [58]	36
3.15	Distribution of number of words in sentences of SNLI corpus [52]	37
3.16	Multi-NLI corpus [14],[58]	38
3.17	Distribution of number of words in MultiNLI corpus	40
3.18	Scitail dataset [30]	40
3.19	distribution of number of words in SCITAIL dataset	42
3.20	Implementation steps [45]	43
3.21	Attend [45]	44
3.22	Compare [45]	45
3.23	Aggregate [45]	46
3.24	Asymmetric attention model [45],[61]	47

3.25	Accuracy at different runs [58]	48
3.26	LSTM model architecture [43],[58]	49
4.1	Accuracy at various Activation functions	52
4.2	Accuracy of different models	53
4.3	Trainable Parameters of different models	54
4.4	Training Time of different models	54
4.5	Analysis with different datasets	55
4.6	Dropout Analysis	57
A.1	Flask app overall interface	66
A.2	Flask app input	66
A.3	Flask app output	66

List of Tables

2.1	Performance of models on SNLI and MultiNLI datasets	21
3.1	Co-occurrence matrix [31]	33
3.2	Co-occurrence matrix [31]	34
3.3	SNLI Corpus distribution	37
3.4	SNLI Labels distribution	38
3.5	Labels distribution of Multi-NLI corpus	40
3.6	Corpus distribution	41
3.7	Corpus distribution	41
4.1	Analysis of sentences for SNLI dataset	58
4.2	Analysis of SNLI sentences	59
4.3	Active Passive sentences [37]	60
4.4	Few correct predictions from MultiNLI test dataset	61
4.5	Few incorrect predictions from MultiNLI test dataset	62

Introduction

“The limits of my language mean the limits of my world.” [66], this quote provides a lucid understanding of the importance of language in this world. The branch of artificial intelligence that deals with language is Natural Language Processing (NLP). More eloquently, it can be stated that NLP is the section of artificial intelligence that enables computers to comprehend and process human languages, the way humans do [64]. One of the subfields of NLP is the task of Recognizing Textual Entailment (RTE). The directional relationship between two sentences is said to be that of entailment, if the first sentence referred as text is true, then the second sentence which is referred to as hypothesis must also be true, example:

- Sentence 1: I teach English to my students.
- Sentence 2: I know to speak English.

The directional relationship between the above sentences is that of entailment because if Sentence 1 is true, Sentence 2 is also true.[15] The directional relationship between the following two sentences is neutral because the sentences are not related.

- Text: I am a good boy
- Hypothesis: This is a pen.

The directional relationship between the following sentences is contradiction, as the sentences are contradictory to each other.

- Text: I am a good boy
- Hypothesis: I am a bad boy.

RTE is a task to categorize or classify the directional relationship between text and hypothesis into three different categories, namely entailment, contradiction and neutral[1],[15]. In [22], Gong et al. refer to the task of RTE as Natural Language Inference (NLI), many other papers also use these two terms interchangeably. Automatic short answer grading system (ASAG) is one of the fields where RTE system can be used to enhance its performance. “An automatic short answer grading system is one which automatically assigns a grade to an answer provided by a student through a comparison with one or more correct answers.” [38] Suppose there is a system for short answer grading that uses semantic textual similarity. Semantic textual similarity is the field of NLP that determines how similar two sentences are.[56] Now, if the question is “Is chlorophyll necessary for photosynthesis?”, and the answer provided by the professor is “Chlorophyll is necessary for photosynthesis” and the answer provided by the student is “Chlorophyll is not necessary for photosynthesis”. These sentences are very similar and the short answer grading system that relies only on semantic textual similarity would find that semantic similarity is high and the student would get full points, even though the answer is wrong. But if the RTE system is used along with semantic textual similarity, then the RTE system will classify it as contradiction, and the student will not get full grades in this case. So, RTE system is used as filter, to enhance the performance of the system.

Apart from short answer grading, RTE is also used in applications like machine translation evaluation, relation extraction, text summarization etc.[15]. There are many approaches used to solve the problem of RTE. The approaches can be categorized into two major domains; they are :

- Non-Deep learning approaches
- Deep learning approaches

In this project, we survey non-deep learning approaches like tree edit distance algorithms, semantic approach, lexical similarity approach, shallow and deep NLP approach, logical inference and deep learning approaches like LSTM (Long Short-Term Memory), tree LSTM, reinforcement learning approaches, convolutional, Neural attention approach. Then we picked two deep learning approaches, decomposable attention model and asymmetric attention model. The main reason behind selecting these approaches is that these approaches attain accuracy comparable to the state of the art with limited number of parameters. In our previous work [58], the idea was to compare LSTM with GRU (Gated Recurrent Units). The model in previous project [58] was chosen was based on the blog of Oreilly media [43]. The major problem with this model was that it was unstable. So, in this project, the model [43] was re-implemented with a few changes, so as to make the model stable. The model was implemented with both LSTM and GRU. Hence, in this project, decomposable attention, asymmetric attention model, LSTM and GRU are compared with respect to accuracy, number of trainable parameters, training time. It was found that decomposable attention model has the best performance and hence was analysed further by testing it with various activation functions like ReLu, sigmoid and tanh. The model's performance was also analysed by varying the dropout value. In this project, the various datasets like SNLI, Multi-NLI, SCITAIL are evaluated on the decomposable attention model. The sentences are analysed to get deep insights about the model and know where and why it fails to perform, and where it performs as per expectation. In chapter 2, we discuss the state of the art techniques for RTE, including both non-deep learning as well as the deep learning methods. In chapter 3, we discuss the concepts related to the approaches, as well as the approaches and the datasets used in this project. In chapter 4, we discuss the various evaluations that were carried out in this project and in chapter 5, the results and conclusions are discussed.

State of the Art

The approaches to solve the problem of RTE can be classified into two main domains:

- Non-Deep learning approaches: Some of the Non-Deep learning approaches discussed in this report are tree edit distance algorithms, semantic approach, lexical similarity approach, shallow and deep NLP approaches.
- Deep learning approaches: The deep learning approaches discussed in this paper are LSTM based approaches, attention-based approaches, convolutional neural network based approach, reinforcement learning based approach, BERT, open AI GPT. The deep learning approaches have obtained state of the art results.

2.1 Non-Deep learning approaches

2.1.1 Tree Edit Distance Algorithms

In [32], Kouylekov et al. proposed a tree edit distance algorithm used on the syntactic representations, also referred to as dependency trees of text as well as hypothesis. This paper was also discussed in our previous work [58]. In this approach, text entails hypothesis if there exists a chain or sequence of transformations that are applied to text such that, the system can obtain a hypothesis with a cost below

the threshold. This approach relies on the assumption that transformation cost is low when the text and hypothesis pairs have a relationship of entailment between them. The overall accuracy obtained by this method is around 55%. This approach is also discussed in our previous paper [58].

A relevant problem we encountered, affecting In this approach, the expressions like “Wal-Mart” and “Wal-Mart inc.” are represented by separate trees, due to this major drawback approximately 30% of the pairs are misclassified. The authors tried to resolve this issue by modifying the order or pattern of given nodes in the tree, but they could not do so because it introduces modifications in the tree edit distance algorithm.

2.1.2 Semantic Approach

In [62], Tatu et al. explains a technique which uses a set of semantic axioms and extracts semantic knowledge from given texts. This system transforms the text and hypothesis pair into semantically improved logical forms .

In this paper, it is stated that logic-based semantic approach is very appropriate for the RTE task. Tatu et al. hypothesize that text semantically entails hypothesis if the meaning logically indicates the meaning of hypothesis. It is to be noted that state of the art semantic parsers try to extract the semantic relations in a given text. In order to fulfil this information, the system needs semantic axioms that increase the extracted knowledge and provides enhanced coverage of the semantics of the text. This information is collected, and it is demonstrated that text entails hypothesis if analogous relations are found between a concept from text and concept from the hypothesis. Analogous concepts refer to the identical words or concepts that are connected by the chain or string of SYNONYMY(refers to synonyms), HYPERNYMY. Hen, eagle, crow, pigeon etc. are related to bird, this relationship is referred to as HYPERNYMY. The results of this paper show that richer semantic connectivity among the text concepts tends to enhance the accuracy of the semantic entailment system.

2.1.3 Lexical Similarity Approach

In [29], Jijkoun et al. explain a system which is based on lexical similarity, that uses two dissimilar word similarity measures. For each word in the given hypothesis, the most alike or similar word in the given text is extracted as per the measure word similarity between text and hypothesis. The sum of the weighted similarity value is found and added to the overall similarity score if a similar word exists. If a similar word doesn't exist, then the given weight of the word is subtracted, so as to penalize the available words in the hypothesis. The threshold value for the ultimate entailment checking is chosen using the development dataset of text as well as hypothesis pairs. The confidence of this system is decided by observing the distance between the given similarity value and its threshold.

max similarity is equal to maximum word similarity between text and hypothesis [29]

$$H_j = \text{hypothesis_sentences} \quad (2.1)$$

$$\text{sim} = \frac{\text{total_similarity}}{\text{total_weight}} \quad (2.2)$$

$$\text{total_similarity}+ = \text{maximum_similarity} * \text{weight}(H_j) \quad (2.3)$$

if similarity is greater than threshold return true, that is entailment , else return false.

$$\text{confidence} = \frac{\text{sim} - \text{threshold}}{1 - \text{threshold}} \quad (2.4)$$

This model has an accuracy of 55.3 percent in test corpus and 61 percent in development corpus of PASCAL-2005 RTE challenge.

2.1.4 Shallow and Deep NLP Methods

In [3], Bos et al. try to combine shallow method that uses word overlap and deep method that uses theorem proving technique. Machine learning is used to aggregate the features extracted from both methods.

The word overlap that occurs between the text and hypothesis are measured by the shallow semantic analysis. The text, as well as hypothesis, are first tokenised and then lemmatised. Each lemma that exists in the hypothesis is given its inverse document frequency by utilizing the web corpus in the form of its weights. This procedure is useful in assigning more importance to words that are less frequent. The shallow technique tries to establish baseline performance and enhance the deep semantic analysis.

Deep semantic analysis is based on the fine-grained semantic representations of each text as well as hypothesis pair generated using wide coverage CCG parser [4]. To find out if it is an entailment or not, they use Vampire [10], which is a theorem prover of first-order logic.

In this system, each text and hypothesis pair is expressed by feature-value vectors which are derived from deep or shallow semantic analysis. A machine learning tool is used after that to get the decision tree model (the decision tree model is derived from the given RTE development set).

In this approach, all features like negation in the text, model size, absolute model size difference, relative domain size difference, etc. were not used. Hence the authors did not expect better results; the experiments that used cross-validation on the development data gave an accuracy of 60 percent on corpus of PASCAL-2005 RTE challenge.

2.2 Deep Learning Approaches

2.2.1 Long Short-Term Memory-Networks(LSTM)

In [7], Cheng et al. intend to model a machine reader that learns to understand text automatically. The term machine reading is in coherence with a variety of tasks such as solving reading comprehension questions [12], relation extraction [18],

ontology learning [48] and textual entailment.

In this approach, a machine reading simulator is used that processes text from left to right, so as to perform shallow analysis with memory and attention. This machine reader uses LSTM architecture, but unlike regular LSTM, a single memory cell is replaced with a memory network. The main idea is to enable adaptive memory usage, in the period of recurrence with neural attention, and thus offers a way to induce relations among tokens to some extent.

This model gave an accuracy of 86.3 percent in SNLI dataset.

In [20], Ghaeini et al. introduce Dependent Reading Bi-directional LSTM (Dr-BiLSTM). The existing NLI (Natural Language Inference) models mostly use attention mechanism, so that they interpret and align, the hypothesis and text. These models use some simple reading mechanisms to encode the hypothesis and text independently. But such intricate work often requires modelling of dependency relationships between hypothesis and text, to prevent the loss of crucial contextual information; such strategies are referred to as dependent reading.

In [55], Rocktschel et al. take into account the dependency aspect of the text as well as hypothesis relationship, but these mechanisms have two limitations:

- The advantages during inference are ignored, and dependency aspects that occur during the encoding stage are considered.
- The dependency aspect in reverse direction is ignored and encoding a hypothesis based on the text is considered.

The Dr-Bi-LSTM model proposed in this paper tends to address these limitations. Given a text t and hypothesis h , the model tries to encode them. In the process of encoding, it also tries to consider their dependence on each other. The soft attention mechanism is then used to get valuable information from these encodings. The soft attention mechanism is also used in decomposable attention based model and is explained in section 2.4

The model consists of an inference stage which uses a dependent reading strategy in two directions, i.e. $t \rightarrow h$ and $h \rightarrow t$, the given sentence representations are passed to this inference stage. The final decision is taken using a multi-layer perceptron in which the aggregated information is passed.

This model gave an accuracy of 89.6 percent on SNLI dataset.

In [6] Chen et al. combine Tree LSTM with Sequential LSTM for Natural Language Inference. The model has the following components:

Soft Alignment

Most of the NLI (Natural Language Inference) use some form of alignment, so as to associate with the relevant subcomponent(the term subcomponent refers to phrases or words) between text and hypothesis. In [45] Parikh et al. use a feed-forward neural network to generate attention weights, but in this paper, they use Bi-LSTM to generate attention weights, and the weights are aligned.

Subcomponent Inference Collection

After alignment has been performed; one can extract the inference information from the given softly aligned sentences. The inference relationship in between the subcomponents of the text and hypothesis sentence is crucial to determine the total inference between the text and hypothesis pair. They used Tree LSTM to achieve this inference relationship. In this approach, they also use three simple matching heuristics like concatenation of two vectors, the difference between them, and one-to-one product. The authors claim that simple heuristics used is better than concatenation used in compositional attention model [45].

Inference Composition

In this layer, they utilize another layer of BiLSTM to model the interaction among the subcomponent inference extracted in subcomponent inference collection layer and get the preliminary output. The preliminary output is fed to the final classifier in order to determine the complete inference relationship and fed to pooling and max-pooling to extract the fixed-length vector.

Extension to Recursive Structures

In this layer, they try to explore the syntax along with semantics for NLI. In this layer, they try to show that involving syntactic parse information is quite

useful when parse information is incorporated in an already existing robust system. They do so by replacing two BiLSTM (one of which is used in soft alignment and the other used in inference composition) by Tree-LSTM. They found that this replacement yielded better performance.

This model gave an accuracy of 88.3 percent on the SNLI dataset.

2.2.2 Reinforcement Learning

In [44], Pan et al. use reinforcement learning in order to optimize a novel objective function that uses the reward. To make proper utilization of the labels information, they use reward in the objective function, which is defined by the property of NLI dataset.

A certain set of words are used by people to explain the discourse relationship between a pair of sentences. The words like “but” , “and” are referred as discourse markers . These discourse markers are deeply connected with the intrinsic relations between the text and hypothesis sentence and are in correspondence with the purport of NLI, such as “but” corresponds to contradiction, and “so” corresponds to entailment.

In Discourse Marker Augmented Network for natural language inference knowledge is transferred from the supervised task, i.e. Discourse Marker Prediction (DMP) [41], so as to form an integrated NLI model. They use a sentence encoder model that tries to learn representations that are obtained from the sentences extracted from the DMP task; after this, they insert the encoder to NLI network. NLI dataset used by them is manually labelled, hence each example from the dataset could get different labels. To make proper use of these various labels, they use reinforcement learning.

The experimental results has shown that this model obtains an accuracy of 89.6 percent on SNLI dataset and 80.3 percent on Multi-NLI dataset.

2.2.3 Reasoning with Entailment

In [55], Rocktschel et al. represent a neural model that is based on LSTMs, it reads the text and hypothesis pairs in one go in order to determine entailment, in contrast to mapping each sentence separately into a semantic space.

In other words, instead of learning sentence representations, they are focusing on neural models which read both texts as well as hypothesis to determine if the relationship between them is that of entailment.

Bi-directional LSTM reads a sequence and reverse of the sequence to improve encoding [24], inspired by that they came up with two-way attention for the task of RTE. The main idea behind this is to utilize the same model in order to attend over the given text t which is conditioned on the hypothesis h , similarly attend over h conditioned on t , simply by swapping the two sequences. This model gives an accuracy of 83.5% on the SNLI dataset.

2.2.4 Tree-Based Convolution and Heuristic Matching

In [39], Mou et al. introduce Tree-Based Convolutional Neural Network (TBCNN) model.

The authors claim that the current trend is to use recurrent/recursive neural networks and convolutional neural network, which yield high performance but are incapable of making full use of the syntactical information present in the sentences. The authors also claim that the aforementioned models are challenging to train because of the long propagation path.

The authors assert that TBCNN gets rid of the aforementioned problems.

TBCNN model is used to extract structural information in the sentences. The authors assert that this property to obtain structural information in sentences is important in NLI. This can be well illustrated by the example that the phrase "driving a car on a highway" can be recognized by TBCNN model using the dependency relations *doobj(driving, car)* and *prepon(driving, highway)*. From the examples, it can be known that TBCNN is better than sequential convolution when word order distortion introduced by modifiers, determinator, etc. are considered. Pooling layer is then applied to aggregate the information. Pooling layer also

helps in semantic compositionality. At last, the 'two sentence' information is integrated by various heuristic matching layers which involve concatenation, as well as element-wise product and difference. Hence they are quite useful when it comes to capturing relationships between pair of sentences, but complexity remains low. Thus the two contributions of this paper are:

- They introduce TBCNN for sentence pair modelling job like NLI.
- They use additional heuristics that enhances the performance of the model while keeping the complexity low and outperforms the contemporary sentence encoding based approaches.

This model gave an accuracy of 82.1 percent on SNLI dataset.

2.2.5 BERT

In [16], Devlin et al. introduce a novel language representation model referred to as BERT (Bidirectional Encoder Representations from Transformers).

The authors state that there are two strategies to enable pre-trained language representations, namely fine-tuning and feature-based, which is applied to downstream tasks. One of the examples of a feature-based approach is ELMO ((Embeddings from Language Models) [47]. ELMO is based on task-specific architectures, which also has pre-trained representations as an additional feature. One of the examples of fine-tuning approach is OpenAI GPT [53]. This approach is based on minimal task-specific parameters, and the pre-trained parameters are fine-tuned by training on the downstream tasks. Both the previous approaches have the same objective function for pre-training, and they use the same unidirectional language models in order to learn the general language representations.

The authors state that the techniques like Open AI GPT[53] and ELMO [47] massively undermine the true potential of pre-trained representations. The dominant limitation is the unidirectional nature of standard language models. This unidirectional nature tends to curb the choice of architectures used during pre-training. In OpenAI GPT left-to-right architecture is used. In this architecture, every token is capable of attending only to the previous tokens of attention layers in the Transformer [63].

In this paper, they show the importance of bi-directional pre-training in the context of language representation. BERT uses masked language models in order to implement a pre-trained bidirectional representation.

They show that pre-trained representation helps to eliminate the requirement of heavily engineered and complicated task-specific architectures. The authors claim that BERT is fine-tuned based representation model which attains state of the art performance on token level tasks as well as a huge suite of sentence-level tasks.

BERT Transformer uses the bidirectional self-attention network.

In the case of BERT, the input representation is unique as it can represent both text sentence pair or a single text sentence as one token sentence. Each token representation corresponds to a summation of its corresponding token, position embeddings as well as segment. This can be well illustrated with figure 2.1 :

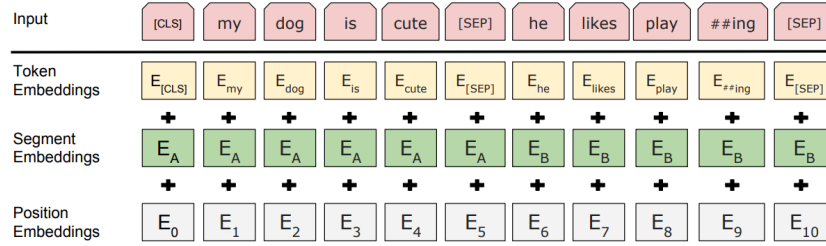


Figure 2.1: BERT input representation [16]

They use WordPiece embeddings [67] that has 30,000 token vocabularies for their representation. They also use learned positional embeddings that support sequence length, which ranges to 512 tokens.

The first token of a sequence is special classification embedding. The final hidden state or the output of the transformer that corresponds to the first token is used to build the aggregate sequence representation necessary for classification tasks. In the case of a non-classification task, this vector is ignored.

They do not use traditional right-to-left or left-to-right language models to train BERT. Instead, they use two new unsupervised prediction tasks to train it.

To train the deep bidirectional representation, they mask 15 percent of input tokens randomly and then predict those masked tokens in their experiments.

Masking allows them to obtain the bidirectional pre-trained model, but it has

two drawbacks. It creates a mismatch between fine-tuning and pre-training, as mask token is not seen while fine-tuning. To get rid of this drawback, they do not replace the masked word with the actual word. Instead, the data generator chooses fifteen percent of the tokens randomly.

The second drawback of using it is that just fifteen percent of the tokens in each batch are predicted which implies that more pretraining steps would be required, such that the model reaches to the point of convergence.

Many tasks like Question Answering, Natural Language Inference, are based on the comprehending the relationship between pair of sentences, and it is not captured by language modelling. In order to train the model capable of understanding sentence relationships, they pre-train the binarized next sentence predictions task that is generated from the monolingual corpus.

To generate an input sentence, two spans of text are taken from the corpus. The first sentence and second sentence receive embedding A and embedding B, respectively. Half of the time B is sentence that follows from A, and half of the time random sentence is chosen. They are taken such that the total length of both the sentence is less than 512 tokens.

The authors claim that recent empirical improvements because of transfer learning have shown that unsupervised pre-training is very important part of language understanding systems. This model gave an accuracy of 83.6 percent on Multi-NLI dataset.

Open AI GPT

In [53], Radford et al. introduce a semi-supervised approach for natural language understanding tasks by combining supervised fine-tuning and unsupervised pre-training. The main idea is to grasp a universal representation that can be used for the tasks of Natural language understanding.

Deep learning techniques require a massive amount of data which is manually labelled to learn. And hence, deep learning approaches get restricted in many domains where such data is not available. The authors of this given paper try to address the problem by developing a model that can extract linguistic information from data which is not labelled by using pre-trained embeddings. The training has

two stages:

Unsupervised Pre-training

$\mathcal{U} = \{u_1, \dots, u_n\}$ is corpus of tokens (unsupervised). Standard language modelling objective is used to maximize the given likelihood

$$L_1(\mathcal{U}) = \sum_i \log P(u_i | u_{i-k}, \dots, u_{i-1}; \Theta)$$

In above equation

- k refers to context window size
- P refers to conditional probability
- Θ refers to neural network parameters

They use multilayer transformer encoder [34] for language modelling.

Multi-headed self-attention operation is applied over the input tokens. Position-wise feedforward layers are used to generate the output distribution for the target tokens.

$$\begin{aligned} h_0 &= UW_e + W_p \\ h_l &= \text{transformer_block}(h_{l-1}) \quad \forall l \in [1, n] \\ P(u) &= \text{softmax}(h_n W_e^T) \end{aligned} \tag{2.5}$$

$$U = (u_{-k}, \dots, u_{-1}) \tag{2.6}$$

- U refers to context vector of tokens.
- n refers to the number of layers
- W_e refers to token embedding matrix
- W_p refers to position embedding matrix

Supervised Fine Tuning

In this step the parameters are adapted with respect to the supervised target task. A labeled dataset $\mathcal{C}$ is taken into consideration where each instance contains a sequence of i/p tokens and label y . The final transformer block's h_l^m is obtained by passing the i/p to pre-trained model. h_l^m is then fed to the added linear o/p layer that has parameters W_y , so as to predict y . This is illustrated by the following equation:

$$P(y|x^1, \dots, x^m) = \text{softmax}(h_l^m W_y) \quad (2.7)$$

The above equation gives the following objection function that needs to be maximized:

$$L_2(\mathcal{C}) = \sum_{(x,y)} \log P(y|x^1, \dots, x^m) \quad (2.8)$$

It is also found that if language modelling is included as an auxiliary objective for the fine-tuning, helped to improve the generalization and accelerate the convergence of the model.

This model gave an accuracy of 89.9% on SNLI dataset.

2.3 Directional Self Attention Network

In [59], Shen et al. propose RNN/CNN-free attention network, which can be utilized for numerous NLP tasks such assentence classification, natural language inference, semantic relatedness, sentiment analysis. This model learns from the sentence encoding, which is considered as a basic module of most of the deep neural network models in the domain of NLP. The motivation behind creating an RNN/CNN free model is listed below:

- In models that use RNNs and its variants, the sequence length has to be fixed. But in case of attention models, there is no such constraint of fixing the sequence length, and hence is more flexible.
- In case of sequential models, the computation cannot be accelerated significantly, but in case of attention models, the computation can be accelerated

significantly by the existing parallel/distributed computing schemes.

The attention model proposed is also novel because of the following features. First, it is multi-dimensional, i.e. the attention with respect to each pair of elements is a vector in which each entry is attention weight that is computed for each feature. Second, it is directional, i.e. it uses one or many positional masks that are used to compute the asymmetric attention weights between two elements. One of the critical features of this model is that it mitigates the shortcoming of attention models in processing temporal order information by applying positional masks on the distribution of attention.

This model is compared with LSTM, and its variants and the authors claim that this model gives state of the art results on SNLI(Stanford Natural Language Inference), MultiNLI(MultiGenre Natural Language Inference), SICK dataset.

This model gave an accuracy of 70.977 percent on Multi-NLI dataset.

2.4 Decomposable Attention Model

In [45], Parikh et al. introduce decomposable attention model. The authors of this paper claim that LSTM and CNN models used for NLI(Natural Language Inference) are computationally expensive and have millions of parameters. Hence the authors came up with the notion of using the attention-based models, which has fewer parameters and reduces the computational cost significantly. Attention based mechanisms are based on the idea that for NLI it is enough to align bits of the local text substructure and aggregate this information. Let us have a look into the following sentences:

- John is in the airport, but because of the storm and heavy rain, the plane has been delayed and he has to wait.
- John is waiting.
- The weather is sunny.

The first sentence is quite complex and it is very challenging to create a compact representation that elaborates its entire meaning. It is quite easy to reach to a conclusion that second sentence entails the first one by aligning John with John

and has to wait with waiting As we can see that the sentence "The weather is sunny ." contradicts with the first sentence, when we align storm and heavy rain with sunny and recognize that they are incompatible. The authors use this intuition to make a simpler model for NLI , that has fewer parameters. The authors claim that this model performs better than the existing complex architectures.

The model uses soft attention mechanism and the entire process can be divided into three steps:

- Attend
- Compare
- Aggregate

2.4.1 Attend

In this step they calculate the unnormalized attention weight which is referred as e_{ij} . The unnormalized attention weight e_{ij} is expressed as: [45]
 $e_{ij} = F(a)^T F(b)$ where $F(a)$ and $F(b)$ are the attention weights obtained by feeding text embedding(a) and hypothesis embedding(b) to feedforward neural network. The unnormalized attention weights are then normalized as follows [45]

$$\begin{aligned}\beta_i &:= \sum_{j=1}^{\ell_b} \frac{\exp(e_{ij})}{\sum_{k=1}^{\ell_b} \exp(e_{ik})} \bar{b}_j \\ \alpha_j &:= \sum_{i=1}^{\ell_a} \frac{\exp(e_{ij})}{\sum_{k=1}^{\ell_a} \exp(e_{kj})} \bar{a}_i\end{aligned}\tag{2.9}$$

2.4.2 Compare

They compare the aligned phrases separately in this step. The text embedding is concatenated with β_i and fed to feedforward neural network to get the comparison vector v1. Similarly, hypothesis embedding is concatenated with α_j and fed to feedforward neural network to get the comparison vector v2.

2.4.3 Aggregate

They calculate the summation of the comparison vectors as shown in this step [45] :

$$\mathbf{v}_1 = \sum_{i=1}^{\ell_a} \mathbf{v}_{1,i} \quad (2.10)$$

$$\mathbf{v}_2 = \sum_{j=1}^{\ell_b} \mathbf{v}_{2,j} \quad (2.11)$$

$\mathbf{v}_1$ and $\mathbf{v}_2$ are concatenated and fed to feed-forward neural network to obtain the final result. The authors claim that this approach performs better than considerably more complex methods used for the task of RTE. This model gave an accuracy of 86.3 percent on SNLI dataset.

2.5 Asymmetric Attention Model

This model is also based on Parikh et al [45] and example notebooks of Spacy [61]. This model intends to demonstrate that in order to classify entailment, only the strength of the association that exists between hypothesis to text is required. This model eliminates the consideration of text to hypothesis vector from the computation. This elimination of text to hypothesis vector reduces the parameter count considerably and enhances the model by reducing the training time. The preliminary calculations until we achieve the attention weights is the same as the previous model. In the previous model, they had the dot product between the normalized attention weights and text embedding (a_j) to get α_j and hypothesis embedding with attention weights to obtain β_i . But in this model, they only calculate α_j and don't calculate β_i . They concatenate α with hypothesis embedding b to obtain comparison vector $\mathbf{v}_2$. $\mathbf{v}_2$ is fed to the feed-forward neural network, and then they calculate the summation of comparison vectors. This summation of comparison vectors is fed to the feed-forward neural network, and the output is fed to softmax layer to obtain the classification.

Models	SNLI(%)	MultiNLI(%)
Long short-term memory- networks for machine reading. [7]	86.3	NA
DR-BiLSTM: Dependent Reading Bidirectional LSTM for Natural Language Inference [20]	89.6	NA
Enhancing and combining sequential and tree lstm for natural language inference [6]	88.3	NA
Discourse Marker Augmented Network with Reinforcement Learning for Natural Language Inference [44]	89.6	80.3
Reasoning about entailment with neural attention [55]	83.5	NA
Improving Language Understanding with Unsupervised Learning [53]	89.9	81.4
Disan: Directional self-attention network for rnn/cnn-free language understanding. [59]	NA	0.70977
Natural Language Inference by Tree-Based Convolution and Heuristic Matching [39]	82.1	NA
A Decomposable Attention Model for Natural Language Inference [45]	86.3	NA
Bert: Pre-training of deep bidirectional transformers for language understanding. [16]	NA	83.6

Table 2.1: Performance of models on SNLI and MultiNLI datasets

2.6 Discussion About the Models

The models that use LSTM in their architecture are computationally very expensive and have high number of parameters. In [45] Parikh et al. state that the model has fewer parameters and attain accuracy comparable to state of the art accuracy. Hence this model was chosen so as to compare how simple and efficient this model is compared to the models using LSTM. The attention mechanism is also used in models like BERT [16], Open AI GPT [53] in their core architecture, hence working with this model would be really helpful in getting proper insights about the core idea of these models as well. So Decompositional attention based model [45] was chosen for analysis.

Methodology

3.1 Neural Network

A neural network can be considered as massively parallel distributed processor which is made of simple processing units and has a natural tendency or propensity to store experiential knowledge and makes it possible for use. Neural network is similar to brain in two respects:

1. Knowledge is gained by the network from the environment by learning process.
2. Interneuron connection strengths which can also referred to as synaptic weights, are utilized to store the obtained knowledge [25]

3.1.1 Neuron

“A neuron is considered as information-processing unit and is elemental to the process of a neural network.” [25] The neuron has these components [25] :

1. It has a set of synapses ,which can also be considered as connecting links that is represented by **weight** . Weights signify the strength of the connection
2. An adder , that is used for summing up the input signals
3. Activation function: Activation is also referred to as squashing function.Activation function limits the amplitude of the neuron’s output.Some of the common activation function used are tanh, sigmoid, ReLu.

4. Bias: The neuron also consists of bias , which is used to either increase or decrease the net input which is fed to the activation function [25]. The impact of bias on the net input can be demonstrated by the Figure 3.1.

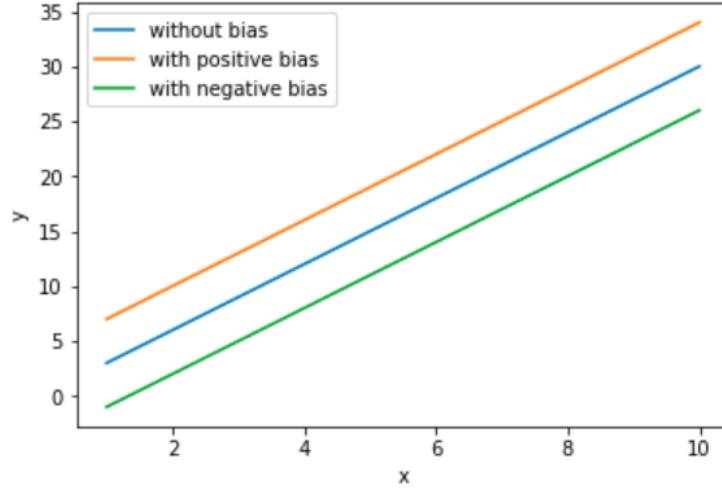


Figure 3.1: Impact of bias on the net input [25]

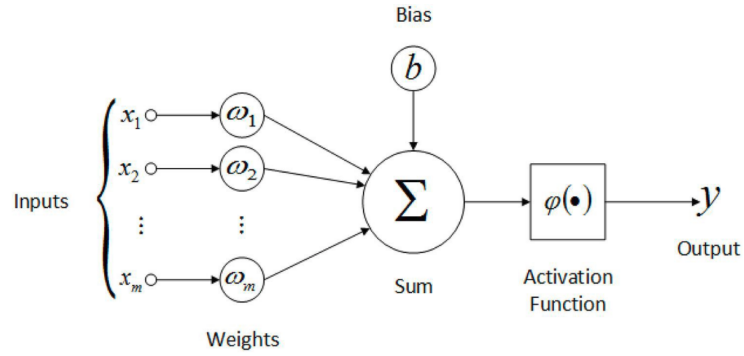


Figure 3.2: Basic Artificial Neuron [13]

As shown in Figure 3.2, a neuron is basically a function approximator , and it can be represented by the equation 3.1 :

$$v_k = \phi\left(\sum_{j=1}^m w_{kj}x_j + b_k\right)[25] \quad (3.1)$$

where ϕ is the activation function, w is the weight and b is the bias [25].

3.1.2 Single-Layer Feedforward Networks

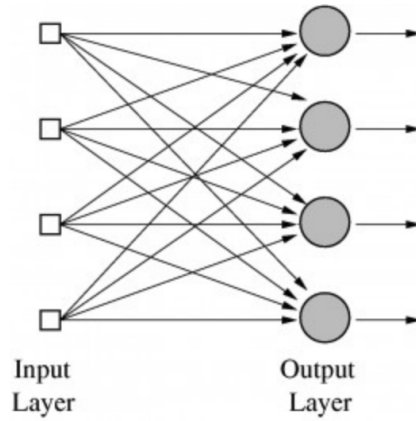


Figure 3.3: Basic Artificial Neuron [28]

The basic architecture of Single layer feed forward neural network can be seen in Figure 3.3. In this network neurons are arranged in the form of layers. This type of network only has input and output layer , and does not have hidden layer. It is also referred to as perceptron. Single layer feed forward neural networks are fast, but cannot model complex functions. They can only model linearly separable functions [13],[51].

3.1.3 Multi-Layer Feedforward Networks

In this network there are input layer, hidden layer and output layer. The hidden layer helps to model more complex functions [5]. The Figure 3.4 shows the simplest possible multilayer feedforward neural network with only two hidden layers.

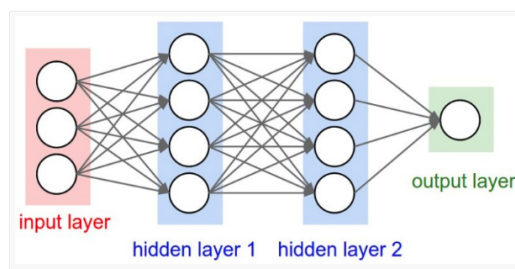


Figure 3.4: Multi-Layer Feedforward Networks [50]

3.1.4 Recurrent Neural Network

Recurrent neural networks have loops in them which allows the information to persist.

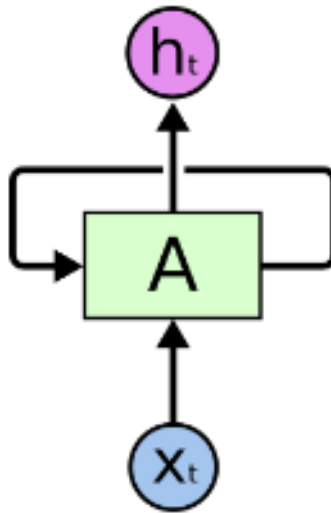


Figure 3.5: Loop in Recurrent Neural Network [42]

In Figure 3.5 ,the neural network looks at input x_t and gives h_t as output. The loop shown in the Figure 3.5 allows information to go from one state of the network to other [42].

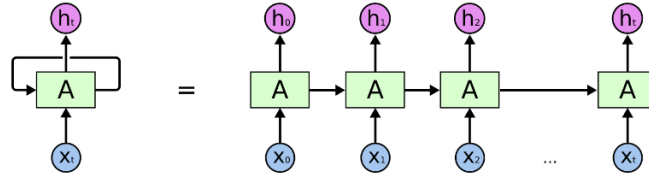


Figure 3.6: unrolled recurrent neural network [42]

RNN should be capable of relating the past information with the present information and make prediction. But they are not as efficient as they should be. If it is needed to predict the ending word in the sentence " the bird flies in the sky " , the gap between the relevant information and place where its required is small, and RNNs learn to utilize the past information.

If it is necessary to predict the ending word in the sentence "I live in Germany and I speak fluent German." The last word is the name of language , and the context of Germany is needed to predict it. But the word Germany is at the back and the gap between the inforamtion which is relevant and the place where it is required is very large , and in such cases where this gap is high. RNNs tend to fail because of its inability to learn to connect relevant information and this problem is referred to as the problem of long term dependency. The Figure 3.6 shows the unrolled recurrent neural network. As seen in the figure there is no mechanism of selectivity , as to which information to keep and which to discard, hence the recurrent neural network suffers from problem of long term dependency.

3.1.5 LSTM

LSTM has also been discussed in our previous work [58]. Recurrent Neural Networks suffer from problem of long term dependency, so Sepp Hochreiter and Jrgen Schmidhuber introduced LSTM [60] in the year 1997. The overall architecture of LSTM is shown in the Figure 3.7.

One of the core parts of the LSTM is the cell state which is represented by the Figure 3.8.

LSTM actually adds or removes information to the cell state, with the help of gates. As the name suggests , gates provides the ability to regulate the flow of

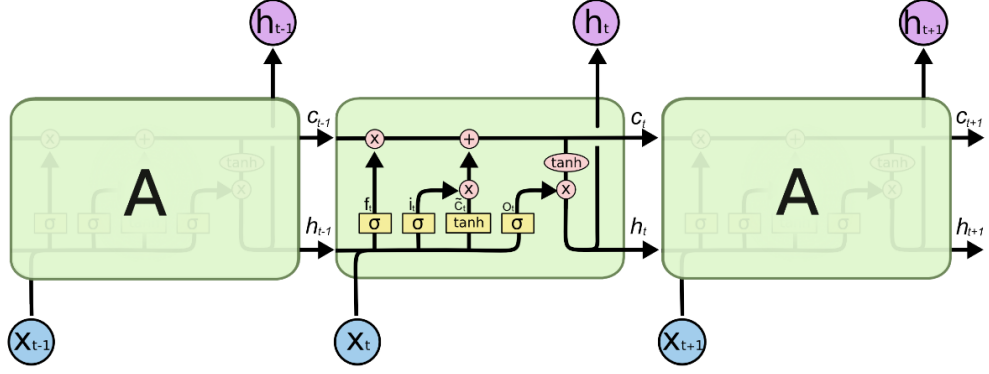


Figure 3.7: LSTM network architecture [42]

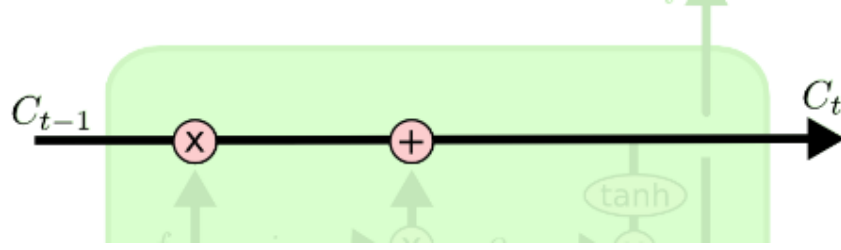


Figure 3.8: Cell state [42]

information. The gates basically consists of two components [42]:

- sigmoid layer
- pointwise multiplication operation

The flow of the information is controlled by the output of sigmoid layer.

One of the gates in LSTM is forget gate. The forget gate looks at the output of previous cell h_{t-1} and current input x_t , and outputs a number that lies between zero and one, for the cell state C_{t-1} . Here, 1 represents that it lets everything through and 0 represents that it doesn't let anything through. The equation for forget gate is given by the equation 3.2 [42]:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (3.2)$$

The next step is in which it is decided whether the new information that needs to be stored in the cell state. It has two parts. Input gate layer decides which values

need to be updated in the cell state. The equation 3.3 represents the input gate [42]:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (3.3)$$

The equation 3.4 represents next cell state [42] :

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (3.4)$$

The input gate is represented by the Figure 3.9 :

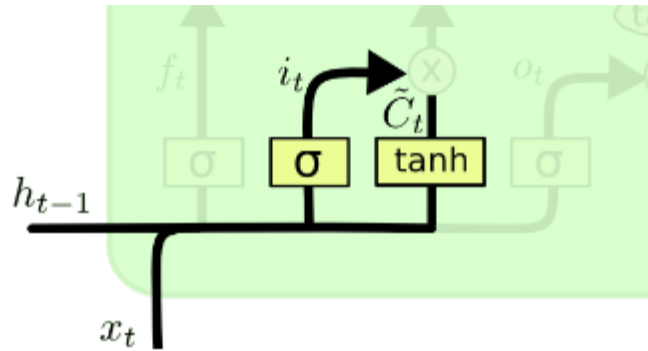


Figure 3.9: Input gate [42]

The old cell state C_{t-1} is updated to new cell state C_t . This is depicted by the Figure 3.10.

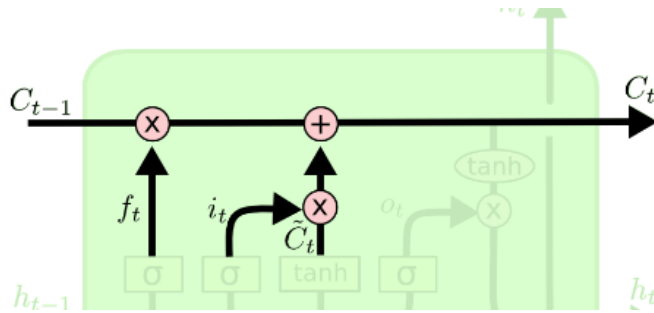


Figure 3.10: cell state [42]

The equation 3.5 represents the state depicted in the Figure 3.10 [42]:

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t \quad (3.5)$$

The output gate decides the final output. The output is based on the cell state . The Figure 3.11 shows the output gate:

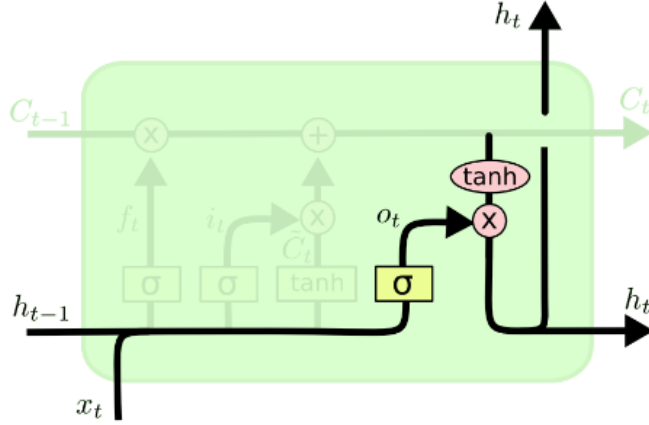


Figure 3.11: output cell state [42]

The equation for the output gate is as follows [42]:

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad (3.6)$$

$$h_t = o_t * \tanh(C_t) \quad (3.7)$$

3.1.6 GRU

Gated Recurrent Unit(GRU) is a variation of LSTM. GRU has also been discussed in our previous work [58].GRU was proposed by Cho et al., 2014 [8]. In GRU forget and input gate are combined to get update gate. The architecture of GRU is shown in the Figure 3.12 :

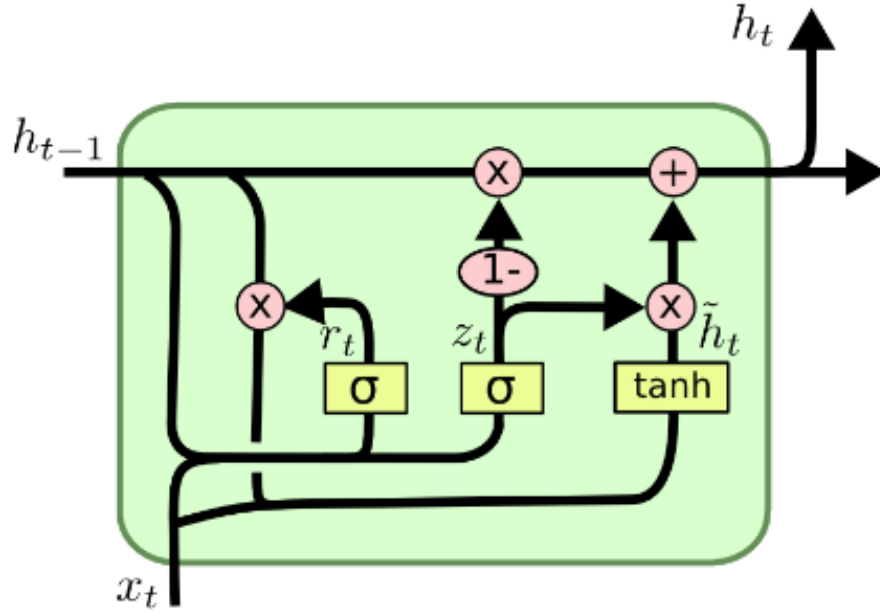


Figure 3.12: GRU architecture [42]

The equations for GRU are as follows [42]:

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t]) \quad (3.8)$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t]) \quad (3.9)$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t]) \quad (3.10)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t \quad (3.11)$$

In the above equations W represents the weight of input, z_t and r_t are update and reset gate vector respectively [42].

3.1.7 Dropout

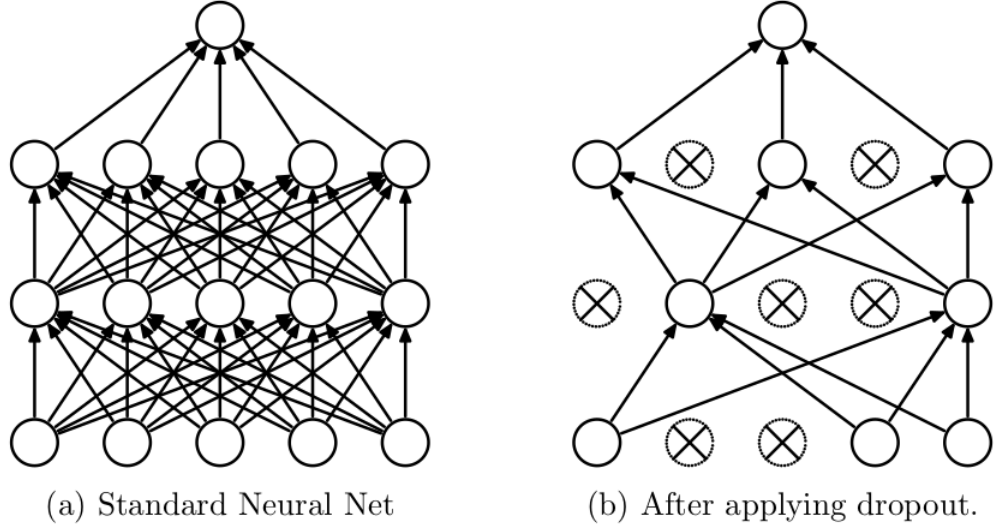


Figure 3.13: Dropout [27]

Deep neural nets has large number of parameters and have proved to be powerful machine learning systems. But deep neural nets suffer from problem of overfitting [27]. Overfitting occurs when a model learns irrelevant details and noise in the data used to train, to an extent that it adversely affects the overall performance of the model [36]. Dropout is a regularization technique used to solve the problem of overfitting. The key idea bheind overfitting is to drop the units randomly during the training. This helps to prevent the units from co-adaptation.

3.1.8 Word Embedding

Word embedding is a representation of words using a particular dimension vector , such that words with similar meanings have similar representation. The word embedding used in this project is GloVe 300 dimension vector [46].The core concept of GloVe comes from co-occurence matrix. “You shall know a word by

the company it keeps.” [19], this is the key idea behind co-occurrence matrix. The table 3.1 shows the co-occurrence matrix for the following three sentences.

- I like reading books.
- I like jogging.
- I enjoy swimming.

counts	I	like	enjoy	reading	books	jogging	swimming	.
I	0	2	1	0	0	0	0	0
like	2	0	0	1	0	1	0	0
enjoy	1	0	0	0	0	0	1	0
reading	0	1	0	0	1	0	0	0
books	0	0	0	1	0	0	0	1
jogging	0	1	0	0	0	0	0	1
swimming	0	0	1	0	0	0	0	1
.	0	0	0	0	1	1	1	0

Table 3.1: Co-occurrence matrix [31]

The non zero co-occurrence matrix is used to train the Glove model.

The prime intuition behind GloVe model is that ratio of word to word co-occurrence probabilities which is obtained from the word co-occurrence matrix can be used to encode some form of meaning. The co-occurrence probabilities are as shown in table 3.2 [46].

Probability and ration	k solid	k gas	k water	k fashion
$P(k ice)$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k steam)$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k ice)P(k steam)$	8.9	8.5×10^{-2}	1.36	0.96

Table 3.2: Co-occurrence matrix [31]

As shown in table 3.2 the co-occurrence porbability of ice with solid is higher than that of ice with gas, and this ratio of probabilities is used to encode some form of meaning. The actual training objective of GloVe vectors is to learn word vectors in such a way that their dot product is equal to the logarithm of these words' relative probabilities of co-occurrence. This is the reason why the GloVe vectors perform very adequately in word analogy tasks [21].

If X represents the word to word co-occurrence counts, and X_{ij} represents the number of times the word j appears for the context or connection of the word i .

The equation 3.12 gives the soft constraints of each word pair :

$$w_i^T w_j + b_i + b_j = \log(X_{ij}) \quad (3.12)$$

w_i represents the vecor of the main word, w_j represents the vector of the context word, b_i and b_j are the scalar biases of the main and given context words.

The equation 3.13 gives the cost function [46].

$$J = \sum_{i=1}^V \sum_{j=1}^V f(X_{ij}) (w_i^T w_j + b_i + b_j - \log X_{ij})^2 \quad (3.13)$$

In the equation 3.14 f is a weighting function that prevents learning only from

very common word pairs.

$$f(X_{ij}) = \begin{cases} \left(\frac{X_{ij}}{x_{max}}\right)^\alpha & \text{if } X_{ij} < x_{max} \\ 1 & \text{otherwise} \end{cases} \quad (3.14)$$

The value of x_{max} is set to 100 and α to be 3/4 in their experiments [46].

3.2 Corpus

The corpus or dataset used in this project are:

- Stanford Natural Language Inference (SNLI) corpus [52]
- Multi-Genre Natural Language Inference (MultiNLI) corpus [65]
- SCITAIL: A Textual Entailment Dataset from Science Question Answering [30]

3.2.1 SNLI

SNLI corpus consists of 570k human-written sentence pairs that are manually labeled. The overall layout of the SNLI corpus can be seen in the figure 3.14.

```
gold_label sentence1_binary_parse sentence2_binary_parse
sentence1_parse sentence2_parse sentence1 sentence2 captionID pairID
label1 label2 label3 label4 label5

neutral ( ( Two women ) ( ( are ( embracing ( while ( holding ( to ( go
packages ) ) ) ) ) . ) ) ( ( The sisters ) ( ( are ( ( hugging goodbye ) ( while
( holding ( to ( ( go packages ) ( after ( just ( eating lunch ) ) ) ) ) ) ) . ) )
(ROOT (S (NP (CD Two) (NNS women)) (VP (VBP are) (VP (VBG
embracing) (SBAR (IN while) (S (NP (VBG holding)) (VP (TO to) (VP (VB
go) (NP (NNS packages)))))) ( . .))) (ROOT (S (NP (DT The) (NNS
sisters)) (VP (VBP are) (VP (VBG hugging) (NP (UH goodbye)) (PP (IN
while) (S (VP (VBG holding) (S (VP (TO to) (VP (VB go) (NP (NNS
packages)) (PP (IN after) (S (ADVP (RB just)) (VP (VBG eating) (NP (NN
lunch)))))))))) ( . .))) Two women are embracing while holding to go
packages. The sisters are hugging goodbye while holding to go packages
after just eating lunch. 4705552913.jpg#2 4705552913.jpg#2r1n
neutral entailment neutral neutral neutral
```

Figure 3.14: first data of SNLI dataset [52] [58]

The SNLI corpus [52] has also been discussed in our previous work [58]. In the Figure 3.14 we can see the following elements :

- *sentence_1_binary_parse* : This is the text sentence
- *sentence_2_binary_parse*: This is the hypothesis sentence
- labels : There are 5 labels , which are the judgement given by 5 different people for a given text & hypothesis pair.
- gold label : The gold label is decided based on the 5 labels, for eg. out of the 5 lables, if there are 3 contradictioons, and 2 neutrals , then the gold label will be taken as contradiction. In some cases when there is not a clear majority for eg. 2 entailments, 2 neutrals and 1 contradictiion, then the gold label is “ _ ”.

Flickr30k corpus [68] was used to collect the premise (premise here refers to the text sentence). Each premise was provided to the worker and they were given the task to crete 3 sentences. The first sentence entails the premise, the second one is neutral and third one contradicts, hence it was ensured that the equal number

of sentence pairs are generated for each label. To validate these sentence pairs a batch of five workers were formed and each batch was provided the same sentence pairs and they labeled it as either entailment or neutral or contradiction [52].

This Figure 3.15 shows the distribution of number of words in sentences of SNLI corpus. The maximum number of words in SNLI dataset is 78.

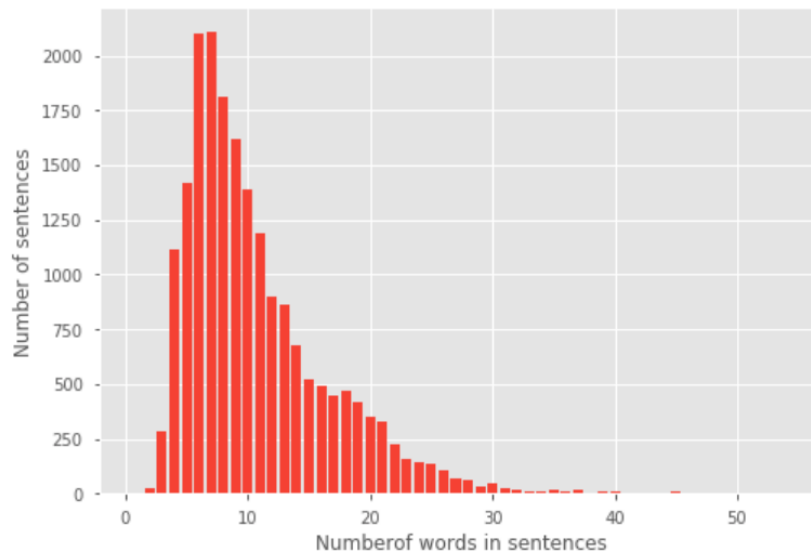


Figure 3.15: Distribution of number of words in sentences of SNLI corpus [52]

The corpus has approximately 570,000 sentences. The corpus is distributed shown in table 3.3 :

Corpus	Number of sentences
Train	550,152
Dev	10,000
Test	10,000

Table 3.3: SNLI Corpus distribution

From the table 3.4 it is known that highest number of sentences fall in the category of entailment, followed by contradiction and neutral has the least number of sentences.

Labels	Number of sentences
Entailment	190113
Neutral	189218
Contradiction	189702

Table 3.4: SNLI Labels distribution

3.2.2 Multi-NLI

The Multi-NLI corpus [14] has also been discussed in our previous work [58]. The overall layout of the corpus is same as SNLI corpus as can be seen in Figure 3.14, but Multi-NLI also provides additional information on genre , the genre could be face-to-face, government ,letters etc. as shown in figure 3.17 .

Met my first girlfriend that way.	FACE-TO-FACE contradiction C C N C	I didn't meet my first girlfriend until later.
8 million in relief in the form of emergency housing.	GOVERNMENT neutral N N N N	The 8 million dollars for emergency housing was still not enough to solve the problem.
Now, as children tend their gardens, they have a new appreciation of their relationship to the land, their cultural heritage, and their community.	LETTERS neutral N N N N	All of the children love working in their gardens.
At 8:34, the Boston Center controller received a third transmission from American 11	9/11 entailment E E E E	The Boston Center controller got a third transmission from American 11.
I am a lacto-vegetarian.	SLATE neutral N N E N	I enjoy eating cheese too much to abstain from dairy.
someone else noticed it and i said well i guess that's true and it was somewhat melodious in other words it wasn't just you know it was really funny	TELEPHONE contradiction C C C C	No one noticed and it wasn't funny at all.

Figure 3.16: Multi-NLI corpus [14],[58]

In [65] Williams et al. claim that SNLI has following drawbacks:

- The text and hypothesis sentences of SNLI are extracted from a single text category/genre image captions , hence are limited to explanations of concrete

visual scenes.

- Sentences fail to capture the notions like temporal reasoning, belief and modality, which are of quite importance to task performance.

To remedy these drawbacks Williams et al. [65] introduce the Multi-NLI corpus. This corpus has 433 thousand sentence pairs that represents written as well as spoken speech in wide range of styles, topics and degree of formality.

Ten freely available sources like Open American National Corpus [25], An American National Corpus: A proposal [13] , The American National Corpus [51] etc were used for the collection of the text sentence.

The text sentences are generated with minimum preprocessing , the sentences which have less than 8 characters are excluded, the irrelevant stuff like bibliographic references ,mathematical formulae are removed manually.

The hypothesis generation technique used in Multi-NLI is similary to that of SNLI, for given text sentence the crowdworker is asked to generate 3 sentences, one that is true whenever the given text is true (Entailment), one which is false when the text sentence is true (Contradiction) and one where neither of the above apply (Neutral).

In order to validate these sentences, each sentence pair was presented to the workers and then they were asked to label the sentence pair as either Entailment or Neutral or Contradiction [65]. The figure 3.17 shows the distribution of the number of words in MultiNLI corpus. From figure 3.17 and 3.15, it can be observed that most of the sentences have words less than 50. So, in our model, the maximum sequence length is set to 50. If something greater than 50 would have been taken then the sequence would be padded with zeros, and the accuracy would be affected, this was observed in the experiments conducted in a similar model in our previous paper [58]. The table 3.5 shows the distribution of labels in the MultiNLI corpus. It can be observed that the labels are evenly distributed.

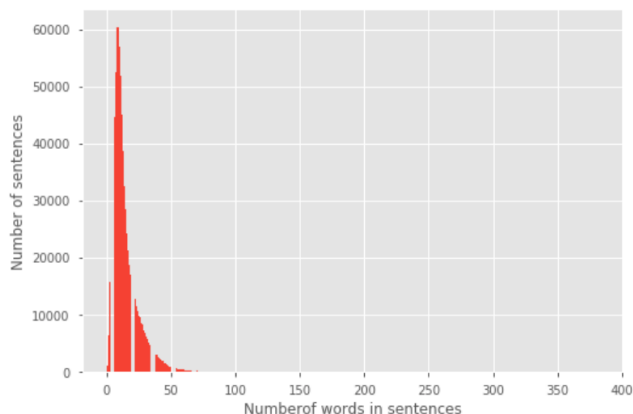


Figure 3.17: Distribution of number of words in MultiNLI corpus

Labels	Number of sentences
Entailment	137841
Neutral	137152
Contradiction	137356

Table 3.5: Labels distribution of Multi-NLI corpus

3.2.3 SCITAIL

Question	Answer	Sentence (Premise)	Q+A as sentence (Hypothesis)	Label
When waves of two different frequencies interfere, what phenomenon occurs?	beating	Beats are the periodic and repeating fluctuations heard in the intensity of a sound when two sound waves of very similar frequencies interfere with one another.	When waves of two different frequencies interfere, beating occurs.	<i>entails</i>
Because trees add water vapor to air, cutting down forests leads to longer periods of what?	drought	During periods of drought, trees died and prairie plants took over previously forested regions.	Because trees add water vapor to air, cutting down forests leads to longer periods of drought.	<i>neutral</i>
What material comprises the sun and other stars, as well as lightning and the northern lights?	plasma	Our Sun and other stars are in the plasma state.	Plasma comprises the sun and other stars, as well as lightning and the northern lights.	–

Figure 3.18: Scitail dataset [30]

SCITAIL dataset [30] was generated for the task to answer multiple choice question for school level science. As shown in the figure 3.18, the question as well as

the correct answer for the multiple choice questions are transformed to an assertive statement, this assertive statement forms the hypothesis H. In order to generate the dataset they used the information retrieval technique to get the relevant text from a huge text corpus of web sentences. They used each of these sentences to develop the premise P. Premise in this paper refers to the text sentence.

Although, P has considerable lexical overlap ,when compared with H, but all P does not entail the assertion in H. They used crowdsourcing to generate the labels for the pair of sentences.

Unlike the regular textual entailment dataset like SNLI , MultiNLI, this dataset has just two labels, entails and neutral, there is no contradiction label. The table 3.6 shows the corpus distribution with respect to training , testing and validation dataset.

Corpus	Number of sentences
Train	23596
Dev	1304
Test	2126

Table 3.6: Corpus distribution

The table 3.7 represents the corpus distribution of SCITAIL dataset with respect to labels Entailment and Neutral.

Corpus	Number of sentences
Entailment	9971
Neutral	16556

Table 3.7: Corpus distribution

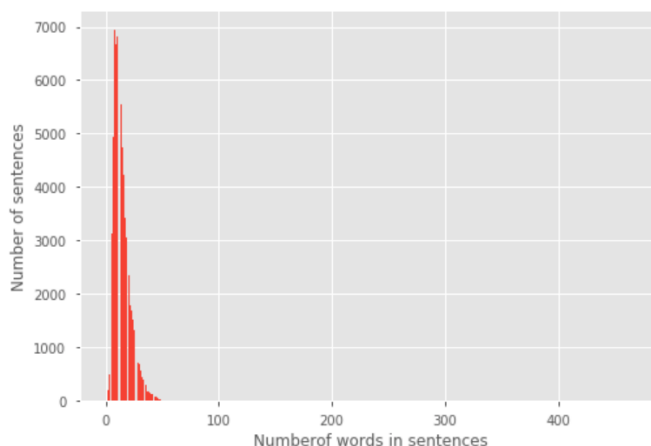


Figure 3.19: distribution of number of words in SCITAIL dataset

The Figure 3.19 represents the distribution of number of words in SCITAIL dataset.

3.3 Implementation

3.3.1 Decomposable Attention Model

The model is based on the paper Parikh et al. [45]. The authors of this paper claim that LSTM and CNN models used for NLI (Natural Language Inference) are computationally expensive and have millions of parameters. Hence, the authors came up with the notion of using the Attention based models, which have fewer parameters and reduce the computational cost significantly. Attention based mechanisms are based on the notion or idea that for NLI, it is enough to align bits of local text substructure and then aggregate this information. Let us have a look into the following sentences:

- John is in the airport, but because of the storm and heavy rain, the plane has been delayed, and he has to wait.
- John is waiting.
- The weather is sunny.

The first sentence is quite complex, and it is very challenging to create a compact representation that elaborates its entire meaning. It is quite easy to reach to a conclusion that the second sentence entails the first one by aligning “John” with “John” and “has to wait” with “waiting”.

As we can see that the sentence “The weather is sunny.” contradicts with the first sentence, when we align “storm and heavy rain” with “sunny” and recognize that they are incompatible.

The authors use this intuition to make a simpler model for NLI, that has fewer parameters.

The authors claim that this model performs better than existing complex architectures.

The implementation is based on the example notebooks of Spacy [61].The implementation has 3 steps in it:

- Attend
- Compare
- Aggregate

It can also be illustrated by following diagram:



Figure 3.20: Implementation steps [45]

Attend

In this step, text embedding and hypothesis embedding are fed to the feed-forward neural network, as shown in the above figure. Instead of taking the dot product

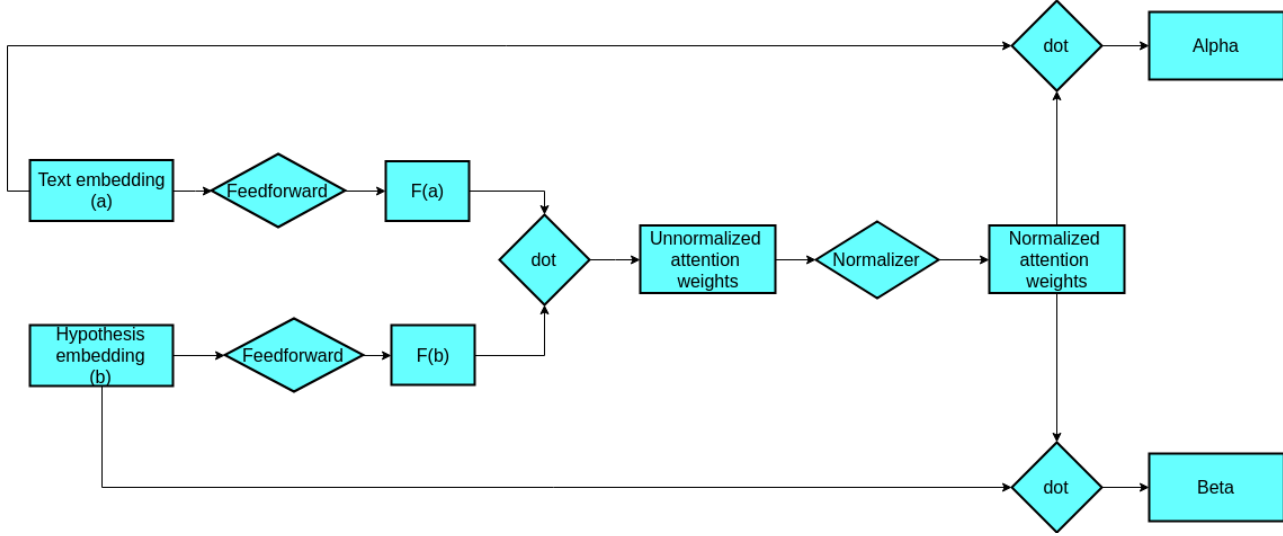


Figure 3.21: Attend [45]

text embedding a and hypothesis embedding b and applying the feed-forward neural network i.e $f(a^T b)$ [45] the feed-forward neural network f is applied to text embedding to get $f(a)$ and to hypothesis embedding to get $f(b)$ and dot product is taken $f(a)^T f(b)$ [45]. Since $f(a^T b)$ is decomposed as $f(a)^T f(b)$ [45], hence this model is referred to as compositional attention model. This decomposition is done to reduce the complexity. The feed-forward neural network uses ReLU activation function. Text embedding when fed to a feed-forward neural network gives $f(a)$ and Hypothesis embedding when fed to the feed-forward neural network gives $f(b)$. There is a dot product between $f(a)$ and $f(b)$, which gives unnormalized attention weights. The unnormalized attention is normalized to get the normalized attention weights. There is a dot product between text embedding and normalized attention weights to get alpha. Similarly, there is a dot product between hypothesis embedding and normalized attention weights to get beta. The above process could be represented by following equations [45]:

$$\begin{aligned}
 \beta_i &:= \sum_{j=1}^{\ell_b} \frac{\exp(e_{ij})}{\sum_{k=1}^{\ell_b} \exp(e_{ik})} \bar{b}_j \\
 \alpha_j &:= \sum_{i=1}^{\ell_a} \frac{\exp(e_{ij})}{\sum_{k=1}^{\ell_a} \exp(e_{kj})} \bar{a}_i
 \end{aligned} \tag{3.15}$$

β_i refers to beta and α_j refers to alpha

e_{ij} is the unnormalized attention weight shown in the above figure. a_i refers to the text sentence, b_j refers to the hypothesis sentence. l_a is the length of text sentence, and l_b is the length of hypothesis sentence.

Compare

In this layer, they compare the aligned phrases. The text embedding and beta are concatenated to get v1. Similarly, hypothesis embedding and alpha are concatenated and fed to the feed-forward neural network to get v2.

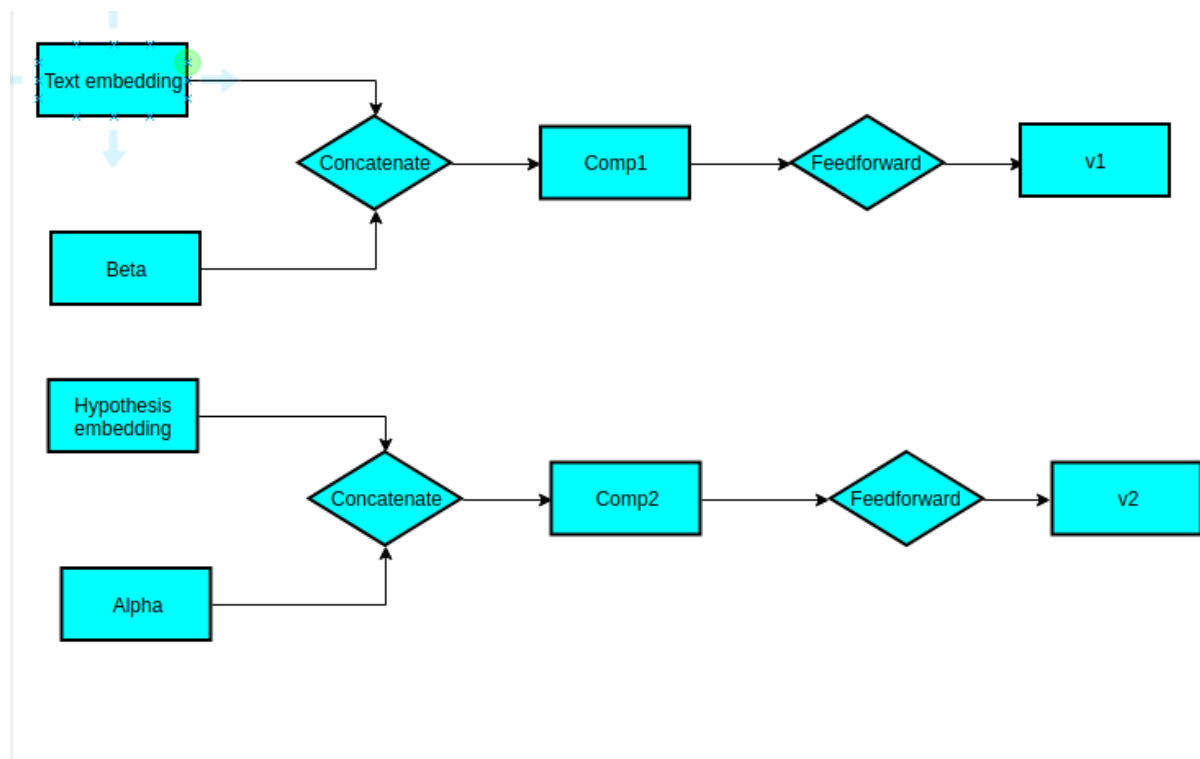


Figure 3.22: Compare [45]

Aggregate

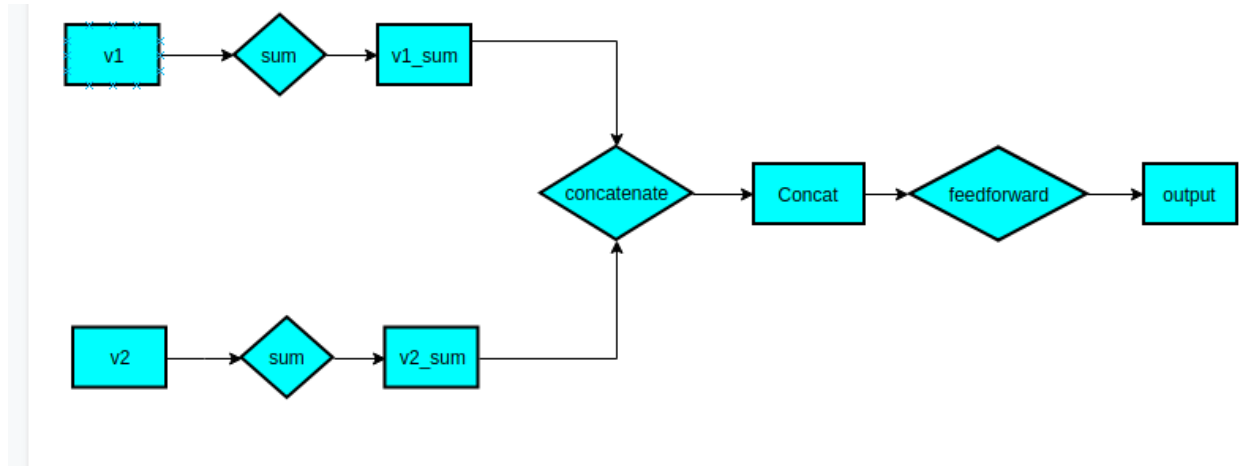


Figure 3.23: Aggregate [45]

`v1` is fed to summing layer to get `v1_sum` and `v2` is fed to summing layer to get `v2_sum`. `v1_sum` and `v2_sum` are concatenated to get `concat`. `concat` is fed to feed forward neural network to get output. The output is then fed to softmax layer to get the final output.

3.3.2 Asymmetric attention model

The asymmetric attention model is explained by the figure 3.24. This model is based on Parikh et al. [45] and example notebooks of Spacy [61].

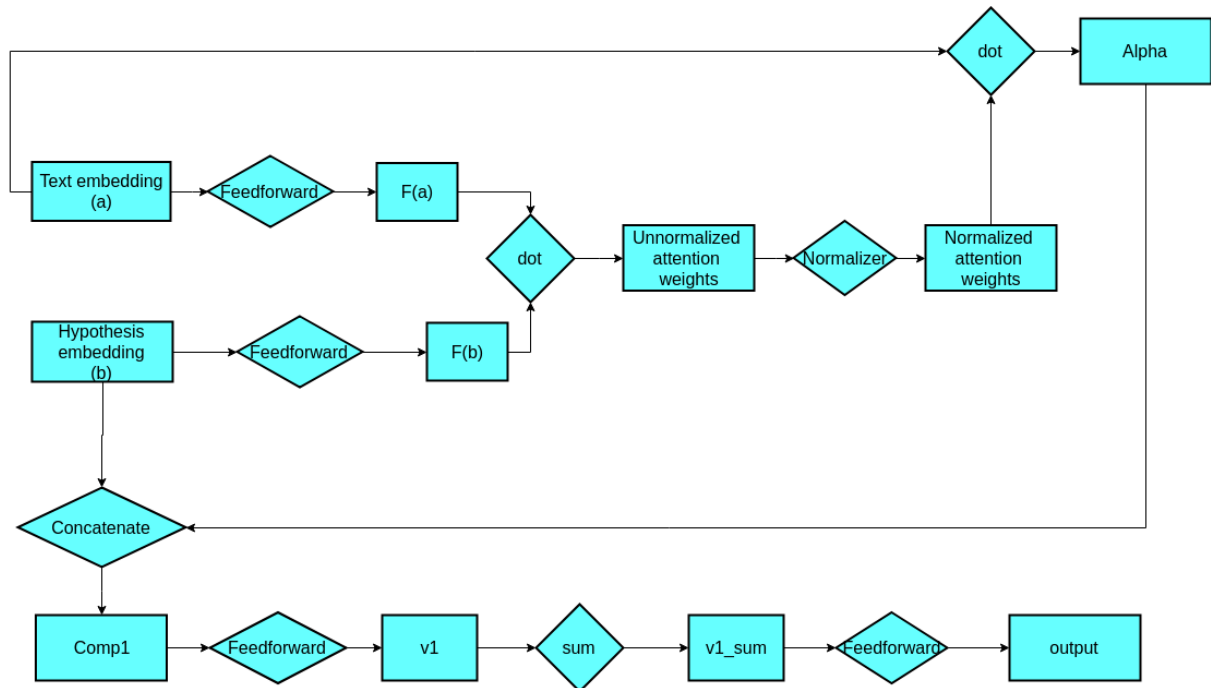


Figure 3.24: Asymmetric attention model [45],[61]

The preliminary steps in this model are same ,till we get attention weights. After that, they carry out the dot product between text embedding and attention weights to get alpha. In the next step the hypothesis embedding and alpha are concatenated, the resultant is referred to as comp1 comp1 is fed to feed forward neural network to get v1. v1 is fed to summing function to obtain v1_sum which is fed to feed forward neural network to get output. The output is then fed to softmax layer to get the final output.

3.3.3 LSTM

In our previous work [58] we had evaluated a LSTM model that was based on the blog Textual Entailment with Tensorflow [43], this implementation had a drawback that the output was not stable. The figure 3.25 shows 5 different runs of the LSTM model.

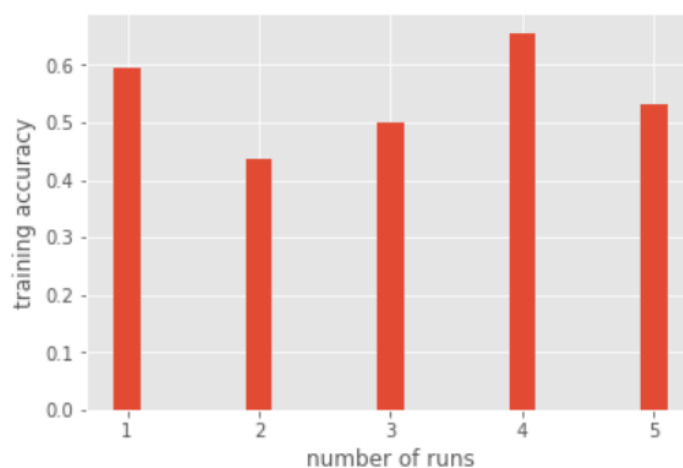


Figure 3.25: Accuracy at different runs [58]

Hence to avoid this anomaly this model was implemented again using Keras library, the Bi-LSTM layers was increased from 64 to 200 and three dense layers was added with a softmax function was added after the Bi-LSTM layers, hence the output of model became stable. The overall architecture or layout of this model can be seen in the figure 3.26.

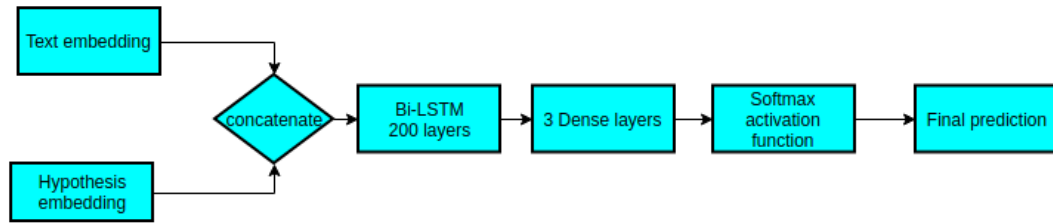


Figure 3.26: LSTM model architecture [43],[58]

The model uses GloVe embeddings of 300 dimensions to convert text and hypothesis sentences to text and hypothesis embeddings. The text and hypothesis embeddings are then concatenated and fed to 200 layers of Bi-LSTM networks. The output of the Bi-LSTM networks is fed to 3 Dense layer and then finally softmax activation function to get the final output.

3.3.4 GRU

The overall architecture of this model is the same as the LSTM model [43]. The only difference is that this model uses GRU instead of LSTM. The main reason for doing so , was to evaluate both of these models in terms of accuracy and complexity.

Evaluation

The three activation functions, namely tanh, ReLu and sigmoid, are evaluated on decomposable attention model, and we try to find which activation function gives the best accuracy. The four different models, namely the decomposable attention model, asymmetric attention model, LSTM and GRU, are evaluated with respect to accuracy, training time and complexity of models. The datasets like SNLI, MultiNLI and SCITAIL are evaluated respectively in decomposable attention model. The impact of dropout on training as well as testing accuracy of the decomposable attention model is analysed. The sentences are fed to decomposable attention model, and the results are recorded, then we analyse the sentences to get deeper understanding of the model.

The parameters used for evaluation are as follows:

- Maximum sequence length : 50
- hidden size : 200
- batch size : 1024
- learning rate : 0.0001
- Embedding dimension : 300

4.1 Activation function

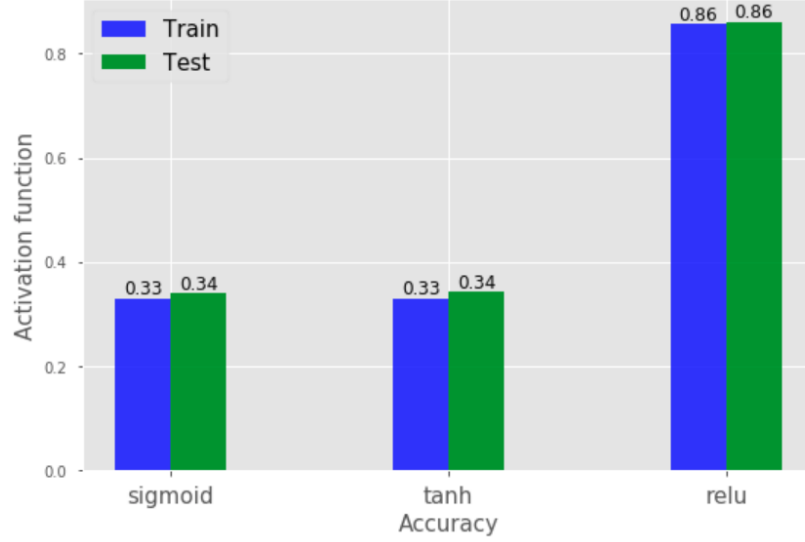


Figure 4.1: Accuracy at various Activation functions

In [2], Bahdanau et al. used tanh activation in their attention network. In [63], Vaswani et al. used softmax activation function in their Scaled Dot-Product Attention network. Hence, in this experiment, three activation functions, namely softmax, tanh and ReLu, are respectively used in the decomposable attention model, and the corresponding accuracies are recorded. The accuracies corresponding to different activation functions can be seen in the figure 4.1. It is found that the model does not learn when softmax and tanh function is used. The model only trains well when ReLu activation function is used. The reason for this could be that when the model uses sigmoid and tanh function, the model suffers from vanishing gradient problem. Hence, it can be concluded that the activation function has a significant impact on the accuracy of the models.

4.2 Evaluation of models

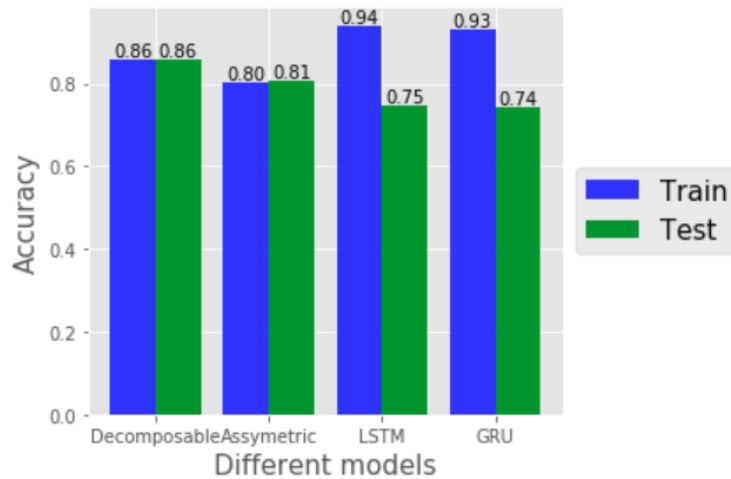


Figure 4.2: Accuracy of different models

In this experiment, the accuracy of decomposable attention model, asymmetric attention model, LSTM as well as GRU model are compared. In the figure 4.2, we can see that the training accuracy of LSTM model is highest, i.e. 94% the testing accuracy is only 75%. So taking testing accuracies into consideration, it is found that decomposable attention model is the best, i.e. it gives an accuracy of 86%. asymmetric attention model gives an accuracy of 81%. LSTM model gives an accuracy of 75%, and GRU gives an accuracy of 74%. Hence from this evaluation, we can conclude that decomposable attention is the best model among these four different models when testing accuracy is taken into account. LSTM model performs better than GRU.



Figure 4.3: Trainable Parameters of different models

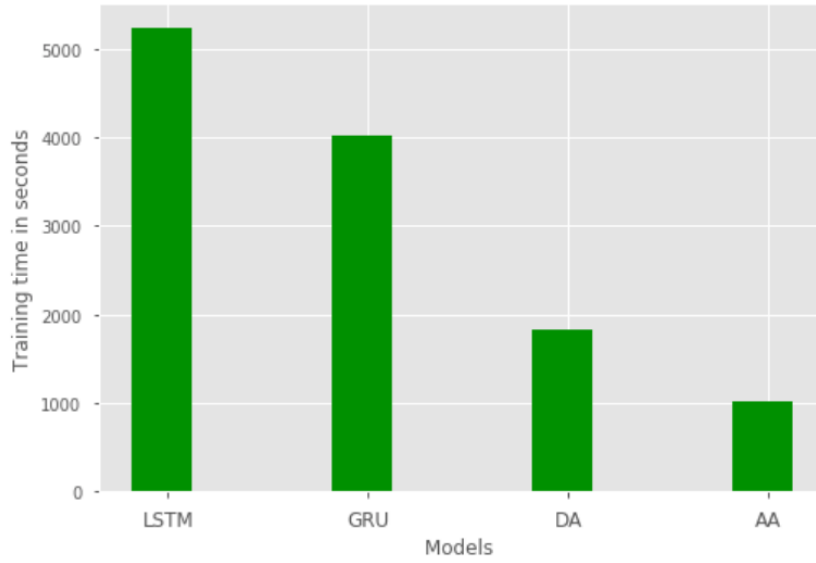


Figure 4.4: Training Time of different models

In the figures 4.3 and 4.4, DA refers to decomposable attention model and AA refers to asymmetric attention model. The figure 4.3 shows the various models with their trainable parameters. LSTM model has the highest number of parameters, followed by GRU. Decomposable attention has the third-highest number of trainable

parameters, and asymmetric attention model has the least number of trainable parameters. From this experiment, we can find that attention-based models have a fewer number of parameters, compared to the LSTM model. The LSTM model taken into consideration is the simplest possible model. Still, the number of trainable parameters is very high compared to decomposable attention model.

The figure 4.4 shows the various models with their training time. If we compare figure 4.3 and figure 4.4 we find that training time and trainable parameters for the models are directly proportional to each other. Thus the training time for LSTM model has highest training time and asymmetric model has least training time.

Decomposable attention model performs better than the LSTM model, by using a fewer number of parameters. Asymmetric attention model has even less number of parameters than decomposable attention model. However, the performance of decomposable attention model is better than asymmetric attention model.

4.3 Datasets

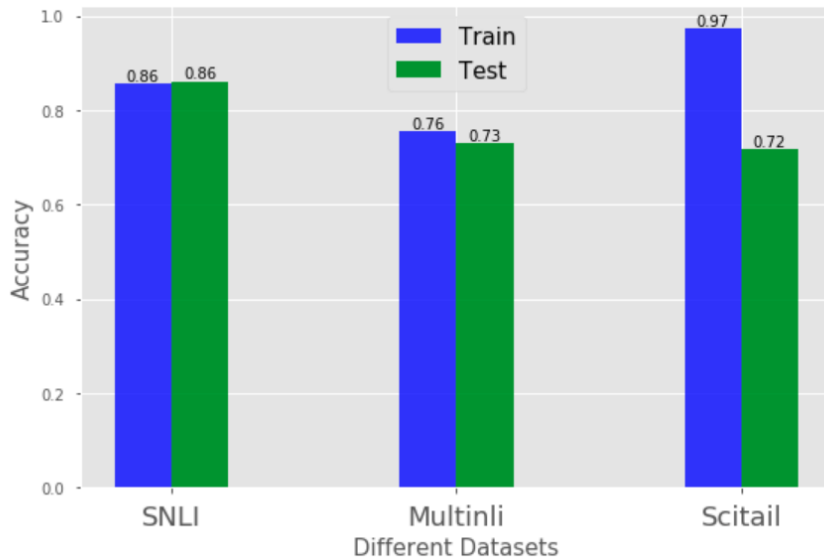


Figure 4.5: Analysis with different datasets

From the previous experiments, it was known that decomposable attention model performs better than other models. Hence, this model was taken into consideration while performing this experiment. In this experiment, decomposable

attention model is trained with SNLI, MultiNLI and SciTail dataset. As seen in figure 4.5, the training accuracy and testing accuracy obtained by using SNLI dataset is 86 percent. The training accuracy obtained using MultiNLI dataset is 76 percent, and testing accuracy obtained is 73 percent. It is seen that the training accuracy obtained using SciTail dataset is 97 percent, but testing accuracy obtained is only 72 percent. Hence considering testing accuracy, we found that model performs best with SNLI dataset. SNLI dataset has the highest number of training sentences, and this could be the reason that the model gives the highest accuracy when trained with SNLI dataset. In the case of the MultiNLI dataset, the training accuracy is more than the testing accuracy. The number of sentences in MultiNLI dataset is less than that of SNLI, and the accuracy is also less. In the case of SciTail dataset, the training accuracy is highest, i.e. 97 percent, but testing accuracy is only 72 percent. It clearly shows that the model suffers from the problem of overfitting when trained with SciTail dataset.

4.4 Dropout

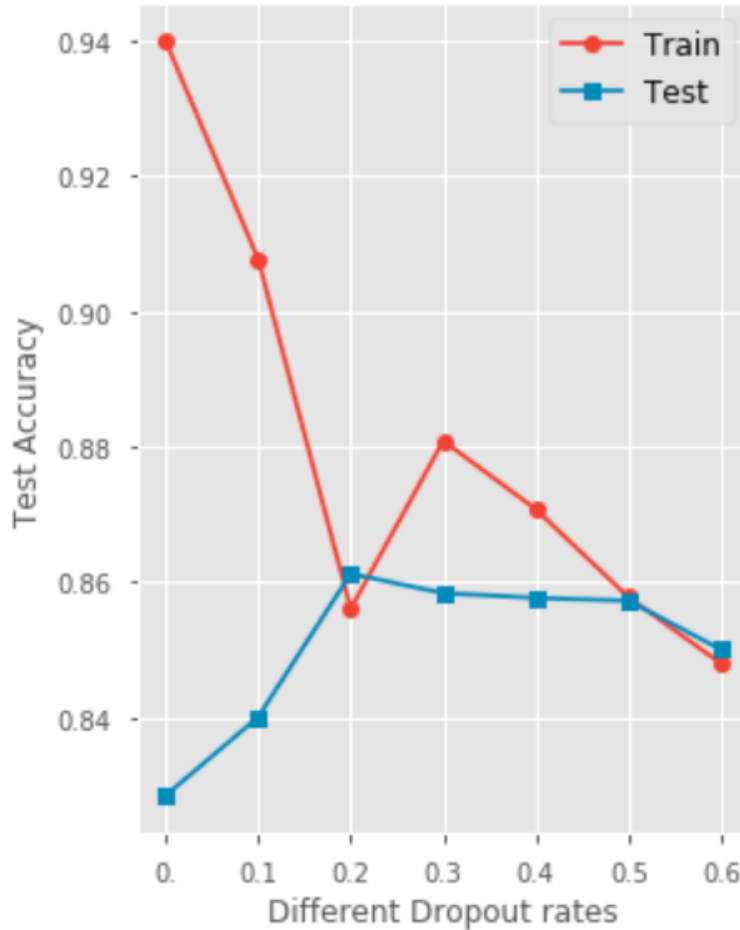


Figure 4.6: Dropout Analysis

This experiment was performed on decomposable attention model. In 2012 Hinton et al. [40] introduced the concept of dropout. This experiment mainly focuses on the impact on dropout on training as well as testing accuracy of the model. As can be seen in the figure 4.6, the training accuracy is highest when there is no dropout, but at the same time, testing accuracy is the least. As dropout is increased from 0 to 0.1, the training accuracy decreases, but the testing accuracy increases. When dropout is 0.2, the testing accuracy is the highest, i.e. 86 percent and training accuracy is the least, i.e. 85.71 percent. When dropout is 0.3, the

training accuracy is 88 percent, and the testing accuracy is 85.84 percent. As the dropout is further increased, the training, as well as testing accuracy both, decrease simultaneously. When dropout is 0.6, the training accuracy and testing accuracy both fall to its minimum value, i.e. 84.8 percent and 85 percent respectively.

4.5 Analysis of Sentences

This experiment was performed on decomposable attention model.

number	Sentence pairs	Known value	Predicted value
1	1. A woman with a green headscarf, blue shirt and a very big grin. 2. The woman is young.	Neutral	Neutral
2	1.A woman with a green headscarf, blue shirt and a very big grin. 2. The woman is very happy.	Entailment	Entailment
3	1. A woman with a green headscarf, blue shirt and a very big grin. 2. The woman has been shot.	Contradiction	Neutral

Table 4.1: Analysis of sentences for SNLI dataset

In the table 4.1 we can see that when the sentences “A woman with a green headscarf, blue shirt and a very big grin.” and “ The woman is young.” are given they are correctly classified as neutral, because there is no relation between the sentences , the word young has no relationship with words of the text sentence. Hence it is correctly classified as neutral. In the same way, for the sentence “ The woman has been shot.”, shot has no relation with the words above; hence, it is wrongly classified as neutral.

number	Sentence pairs	Known value	Predicted value
1	1. The two young girls are dressed as fairies, and green are playing in the leaves outdoors. 2. Girls are playing outdoors.	Entailment	Entailment
2	1. The two young girls are dressed as fairies, and are playing in the leaves outdoors. 2. Two girls play dress up indoors.	Contradiction	Contradiction
3	1. The two young girls are dressed as fairies, and are playing in the leaves outdoors. 2. The two girls play in the Autumn.	Entailment	Neutral

Table 4.2: Analysis of SNLI sentences

As explained in the description of the model, the model first aligns, then compares and then aggregates to give the prediction. Hence in the first sentence of the table 4.2, “playing” is aligned with “playing”, and “outdoors” with “outdoors”, therefore it is classified as entailment. Whereas in the second sentence, the words “outdoors” is aligned with “indoors” and since the words are contradictory, hence the sentences are classified as contradiction. Similarly “outdoors” has no relationship with “Autumn”. Consequently, the model classifies it as neutral, even though it is labeled as entailment.

4.5.1 active-passive sentences

number	Sentence pairs	Known value	Predicted value
1	1. Harry ate six shrimp at dinner. 2. At dinner, six shrimp were eaten by Harry.	Entailment	Entailment
2	1.Beautiful giraffes roam the savannah. 2. The savannah is roamed by beautiful giraffes.	Entailment	Entailment
3	1. Sue changed the flat tire. 2. The flat tire was changed by Sue.	Entailment	Entailment
4	1. We are going to watch a movie tonight. 2. A movie is going to be watched by us tonight.	Entailment	Entailment
5	1. I ran the obstacle course in record time. 2.The obstacle course was run by me in record time..	Entailment	Entailment

Table 4.3: Active Passive sentences [37]

In the SNLI as well as MultiNLI dataset , we came across some active-passive sentence pairs. Hence, an experiment was conducted precisely on active-passive sentence pairs. For this purpose a list of active passive sentences were taken from Your Dictionary website [37].The sentences were given the label entailment and were tested in decomposable attention model. All the sentences were correctly classified.

In the table 4.3, “Harry” is aligned with “Harry”, “ate” with “eaten”, “six shrimp” with “six shrimp” ,“at dinner” with “At dinner”, hence the sentences were classified as entailment. Similarly in the rest of the sentences as well the words are aligned respectively. Since in active-passive sentences, only a few extra words are added and the arrangement of words are different, but the sentences are similar.Hence, the sentences are correctly classified by the model.

number	Sentence pairs	Known value	Predicted value
1	1. yeah i i think my favorite restaurant is always been the one closest you know the closest as long as it's it meets the minimum criteria you know of good food. 2. My favorite restaurants are always at least a hundred miles away from my house.	Contradiction	Contradiction
2	1.i don't know um do you do a lot of camping. 2. I know exactly	Contradiction	Contradiction
3	1. well that would be a help i wish they would do that here we have got so little landfill space left that we're going to run out before the end of this decade and it's really going to be. 2. We have plenty of space in the landfill.	Contradiction	Contradiction
4	1. yeah i know and i did that all through college and it worked too. 2. I did that all through college but it never worked .	contradiction	contradiction

Table 4.4: Few correct predictions from MultiNLI test dataset

In the table 4.4 first pair of sentences "my favorite restaurant" is and sentence pair "I don't know" is aligned with "I know", since "I know" and "I don't know" are contradictory hence, the sentences are classified as contradiction. In the second sentence pair "I don't know" is aligned with "I know" and since they are contradictory, hence the sentences are classified as contradictory. In third sentence pair "We have" is aligned with "We have", "little space" is aligned with "plenty of space" and "landfill space" is aligned with "space in the landfill", since "little space" is contradictory to "plenty of space", hence the sentences are classified as contradiction. In fourth pair of sentences, "i did that all through" is aligned with college" I did that all through college" and "it worked too" is aligned with "it never worked", since "it worked too" is contradictory to "it never worked", hence

the sentences are classified as contradiction.

Number	Sentence pairs	Known value	Predicted value
1	1. and uh as a matter of fact he's a draft dodger 2. They dodged the draft, I'll have you know.	Entailment	Neutral
2	1.Look, there's a legend here. 2. See, there is a well known hero here.	Entailment	Neutral
3	1.She has exchanged a hollow life for a heightened life, and has tried to comprehend all its turns, get its possibilities. 2. She has chose to live a hollow life.	Contradiction	Entailment
4	1. Tuppence rose. 2.Tuppence floated into the air.	Neutral	Entailment

Table 4.5: Few incorrect predictions from MultiNLI test dataset

In the first pair of sentence we can see the sentences are not similar at all. The first sentence has only one subject i.e. “he”, but in second sentence there are two subjects “They” and “I”, the sentences don’t have a relation as such, so it is wrongly classified as neutral, instead of entailment. In second sentence “Look, there’s” gets aligned with “ See, there is”, but “a legend here” gets aligned with “a well know hero”, and the model fails to find the relation between “a legend” and “a well known hero” and hence the sentence pair is classified as neutral, instead of entailment. In third pair of sentence , “She has” gets aligned with “She has”, and “hollow life” with “hollow life”, hence the sentence pair is classified as entailment instead of contradiction. In fourth sentence pair , “Tuppence” gets aligned with “Tuppence”, and “rose” is aligned with “floated into the air”, hence the sentences are classified as neutral instead of entailment, as model fails to find the relation between “rose” and “floated in air”.

Conclusions

- In the first experiment the three activation functions namely sigmoid, tanh and ReLu were tested on decomposable attention model and it was found that model suffers from vanishing gradient problem when sigmoid and tanh activation function is used. Hence we can conclude that ReLu activation function can be used to avoid the vanishing gradient problem. This experiment also helps us to come to a conclusion that activation function has significant importance in the overall performance of the model.
- In the second experiment, the decomposable attention, asymmetric attention, LSTM and GRU models are tested using SNLI dataset. It was found that the decomposable attention model has the best accuracy among the four models. The decomposable attention model has the least number of parameters and requires less training time compared to LSTM and GRU models. Hence from this experiment, we can conclude that attention models perform better than LSTM and GRU models.
- In the second experiment it was also found that asymmetric attention model has a fewer number of parameters and requires lesser training time compared to decomposable attention model, but the accuracy of asymmetric model is lesser than that of decomposable attention model. Hence it can be concluded that decomposable attention model has fewer parameters, requires lesser training time and has better accuracy.

-
- In third experiment, the decomposable attention model was trained on SNLI, MultiNLI and SciTail dataset. It was found that the model trained on SNLI dataset has the highest accuracy. SNLI dataset has the highest number of training sentences; hence, it can be concluded that the number of sentences in the training dataset has significant impact on the overall accuracy of the model.
 - In the fourth experiment, it was found that the performance was highest when the dropout rate is 0.2. Hence it can be concluded that optimum value of dropout rate should be considered to ensure maximum accuracy. It can also be concluded that the dropout rate has a significant impact on the overall performance of the model.
 - In the fifth experiment, the various sentences were analysed, and it was found that when the sentences are similar for eg. in case of active passive sentence pairs, then the model correctly classifies the sentences. The key idea is that when the sentences are aligned and when it finds a proper relation it classifies it, but in some cases the sentences are complex, and this strategy of aligning and predicting fails and the model is not able to make correct predictions.

A

Deployment of Deep learning model in development server

In our previous project [58], one of the tasks was to deploy the model in the development server, but due to time constraints, it was not done. In this project, the decomposable attention model was deployed in the development server. The code for the implementation is based on the blog “Towards Data Science” [57], which is related to spam message filtering using machine learning. First of all, the model is trained, and the model is saved in json format. The weights file is also saved in h5 format. The deployment was done using flask. Flask is lightweight WSGI (Web Server Gateway Interface) web application framework [49]. The model and the weights are loaded in the flask application. The flask application provides two inputs, one for the text sentence and other for the hypothesis sentence. The text and hypothesis sentences are converted to text and hypothesis embedding; the text and hypothesis embedding are used to predict the directional relationship between the sentences, i.e. either entailment or neutral or contradiction. The prediction is then displayed as the output of the model in the application. Figure A.1 shows the overall interface of the app.

The figure A.2 shows the input given to the flask app input. As can be seen in the figure A.2 ,two inputs are given “He is a good boy” and “He is a bad boy” and the output is “contradiction” as shown in the figure A.3.



Figure A.1: Flask app overall interface



Figure A.2: Flask app input

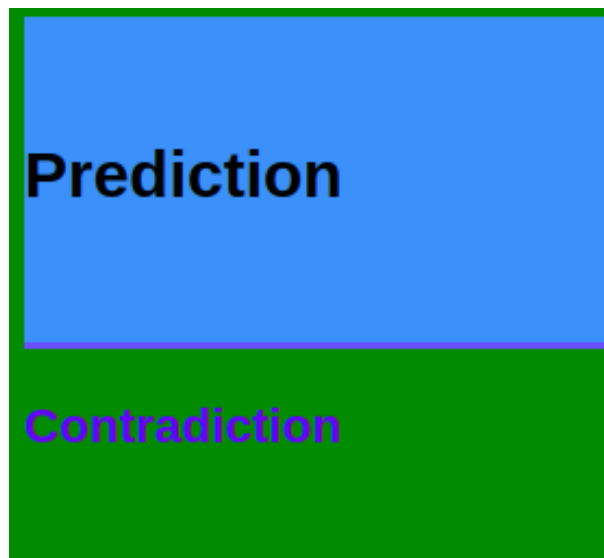


Figure A.3: Flask app output

Bibliography

- [1] PASCAL Recognizing Textual Entailment Challenge (RTE-7) at TAC 2011. <https://tac.nist.gov//2011/RTE/>. Accessed on 30 April 2019.
- [2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *ICLR*, 2015.
- [3] Johan Bos and Katja Markert. Combining shallow and deep nlp methods for recognizing textual entailment. In *PASCAL*, 2005.
- [4] Johan Bos, Stephen Clark, Mark Steedman, James R. Curran, and Julia Hockenmaier. Wide-coverage semantic representations from a ccg parser. In *20th International Conference on Computational Linguistics; Geneva, Switzerland*, 2004.
- [5] brilliant.org. <https://brilliant.org/wiki/feedforward-neural-networks/>. Accessed on 17 June 2019.
- [6] Qian Chen, Xiaodan Zhu, Zhenhua Ling, Si Wei, and Hui Jiang. Enhancing and combining sequential and tree lstm for natural language inference. In *arxiv.org*, 2016.
- [7] Jianpeng Cheng, Li Dong, and Mirella Lapata. Long short-term memory-networks for machine reading. In *Cornell University*, 2016.
- [8] Kyunghyun Cho, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. In *EMNLP 2014*, 1997.
- [9] Philipp Cimiano, Christina Unger, and John McCrae. Ontology-based interpretation of natural language. In *Morgan and Claypool Publishers*, 2014.

- [10] K. Claessen and N. Sorensson. New techniques that improve mace-style model finding. In *In Model Computationa - Principles, Algorithms, Applications (Cade-19 Workshop), Miami, Florida*, 2003.
- [11] Peter Clark and Phil Harrison. Recognizing textual entailment with logical inference. In *PASCAL*, 2005.
- [12] Peter Clark, Phil Harrison, and Niranjan Balasubramanian. A study of the knowledge base requirements for passing an elementary science test. In *ACM*, 2013.
- [13] Medium Corporation. <https://medium.com/@jayeshbahire/the-artificial-neural-networks-handbook-part-4-d2087d1f583e>. Accessed on 17 June 2019.
- [14] The Multi-Genre NLI Corpus. <https://www.nyu.edu/projects/bowman/multinli/>. Accessed on 7 August.
- [15] Ido Dagan, Dan Roth, Mark Sammons, and Fabion Zenzotto. *Recognizing Textual Entailment Models and Applications*. Morgan & Claypool Publishers, 2003.
- [16] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *arxiv.org*, 2018.
- [17] Your Dictionary. <https://examples.yourdictionary.com/examples-of-active-and-passive-voice.html>. Accessed on 30 April 2019.
- [18] Anthony Fader, Stephen Soderland, and Oren Etzioni. Identifying relations for open information extraction. In *Acl*, 2013.
- [19] J.R. Firth. Papers in linguistics 193451. 1957.
- [20] Reza Ghaeini, Sadid A. Hasan², Vivek Datla, Joey Liu, Kathy Lee, Ashequl Qadir, Yuan Ling², Aaditya Prakash², Xiaoli Z. Fern¹, and Oladimeji Farri².

- Dr-bilstm: Dependent reading bidirectional lstm for natural language inference. In *arxiv.org*, 2018.
- [21] GloVe. <https://nlp.stanford.edu/projects/glove>. Accessed on 17 June 2019.
- [22] Yichen Gong, Heng Luo, and Jian Zhang. Natural language inference over interaction space. In *conference paper at ICLR 2018*, 2018.
- [23] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. The MIT Press, 2016.
- [24] Alex Graves and Jurgen Schmidhuber. Framewise phoneme classification with bidirectional lstm and other neural network architectures. In *arxiv.org*, 2005.
- [25] Simon Haykin. *Neural Networks and Learning Machines*. PEARSON Prentice hall, 1999.
- [26] Andrew Hickl, Jeremy Bensley, John Williams, Kirk Roberts, Bryan Rink, , and Ying Shi. Recognizing textual entailment with lccs groundhog system. In *PASCAL*, 2006.
- [27] Nitish Srivastava Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. 2014.
- [28] <http://www.andreykurenkov.com>. <http://www.andreykurenkov.com/writing/ai/a-brief-history-of-neural-nets-and-deep-learning/>. Accessed on 8 September 2019.
- [29] Valentin Jijkoun and Maarten de Rijke. Recognizing textual entailment using lexical similarity. In *PASCAL*, 2005.
- [30] Tushar Khot, Ashish Sabharwal, and Peter Clark. SciTail: A textual entailment dataset from science question answering. In *AAAI*, 2018.
- [31] M. Korniyenko and S. Samson. <http://www.sfs.uni-tuebingen.de/~ddekok/dl4nlp/glove-presentation.pdf>. Accessed on 17 June 2019.

- [32] Milen Kouylekov and Bernardo Magnini. Recognizing textual entailment with tree edit distance algorithms. In *PASCAL*, 2005.
- [33] Dekang Lin and Pantel. Discovery of inference rules for question answering. In *Natural Language Engineering* 7, 2001.
- [34] Peter J. Liu, Mohammad Saleh, Etienne Pot, Ben Goodrich, Ryan Sepassi, ukasz Kaiser, and Noam Shazeer. Generating wikipedia by summarizing long sequences. In *arxiv.org*, 2018.
- [35] Xiaodong Liu, Pengcheng He, Weizhu Chen, and Jianfeng Gao. Multi-task deep neural networks for natural language understanding. In *arxiv.org*, 2019.
- [36] Machine Learning Mastery. <https://machinelearningmastery.com/overfitting-and-underfitting-with-machine-learning-algorithms/>, . Accessed on 8 September 2019.
- [37] Machine Learning Mastery. <https://examples.yourdictionary.com/examples-of-active-and-passive-voice.html>, . Accessed on 8 September 2019.
- [38] Michael Mohler and Rada Mihalcea. Text-to-text semantic similarity for automatic short answer grading. In *Conference of the European Chapter of the ACL*, 2009.
- [39] Lili Mou, Rui Men, Ge Li, Yan Xu, Lu Zhang, Rui Yan, and Zhi Jin¹. Natural language inference by tree-based convolution and heuristic matching. In *arxiv.org*, 2016.
- [40] Improving neural networks by preventing co-adaptation of feature detectors. Geoffrey e. hinton and nitish srivastava and alex krizhevsky and ilya sutskever and ruslan r. salakhutdinov. 2012.
- [41] Allen Nie, Erin D. Bennett, and Noah D. Goodman. Dissent: Sentence representation learning from explicit discourse relations. In *arxiv.org*, 2017.
- [42] Christopher Olah. <http://colah.github.io/posts/2015-08-Understanding-LSTMs/>. Accessed on 17 June 2019.

- [43] Oreilly. <https://www.oreilly.com/learning/textual-entailment-with-tensorflow>. Accessed on 23 June 2019.
- [44] Boyuan Pan, Yazheng Yang, Zhou Zhao, Yueting Zhuang, Deng Cai, and Xiaofei He. Discourse marker augmented network with reinforcement learning for natural language inference. In *acl*, 2018.
- [45] Ankur P. Parikh, Oscar Tckstrm, Dipanjan Das, and Jakob Uszkorei. A decomposable attention model for natural language inference. In *arxiv.org*, 2016.
- [46] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. Glove: Global vectors for word representation. In *conference on empirical methods in natural language processing (EMNLP)*, 2014.
- [47] Matthew E. Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. Deep contextualized word representations. In *arxiv.org*, 2018.
- [48] Hoifung Poon and Pedro Domingos. Unsupervised ontology induction from text. In *Acl*, 2010.
- [49] The Pallets Projects. <https://palletsprojects.com/p/flask/>. Accessed on 8 September 2019.
- [50] pyimagesearch.com. <https://www.pyimagesearch.com/2016/09/26/a-simple-neural-network-with-python-and-keras/>. Accessed on 17 June 2019.
- [51] Quora. <https://www.quora.com/What-are-the-advantages-and-disadvantages-of-the-sing>. Accessed on 17 June 2019.
- [52] Samuel R. Bowman, Gabor Angeli, Christopher Potts, and Christopher D. Manning. A large annotated corpus for learning natural language inference. In *Cornell University Library*, 2015.
- [53] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. In *arxiv.org*, 2018.

- [54] Tim Rocktaschel, Edward Grefenstette, Karl Moritz Hermann, Tomas Koisk y, and Phil Blunsom. Reasoning about entailment with neural attention. In *arxiv.org*, 2016.
- [55] Tim Rocktschel, Edward Grefenstette, Karl Moritz Hermann, Tom Koisk, and Phil Blunsom. Reasoning about entailment with neural attention. In *arxiv.org*, 2015.
- [56] Sebastian Ruder. http://nlpprogress.com/english/semantic_textual_similarity.html. Accessed on 8 September 2019.
- [57] Towards Data Science. <https://towardsdatascience.com/develop-a-nlp-model-in-python-deploy-it-with-flask-step-by-step-744f3bdd777>. Accessed on 8 September 2019.
- [58] Ramit Sharma, Professor Dr. Paul G. Ploeger, and Msc. Deebul Nair. Recognizing textual entailment : A comprehensive evaluation of the existing state of the art techniques. In *R&D report hbrs*, 2018.
- [59] Tao Shen, Jing Jiang, Tianyi Zhou, Shirui Pan, Guodong Long, and Chengqi Zhang. Disan: Directional self-attention network for rnn/cnn-free language understanding. In *arxiv.org*, 2017.
- [60] Long short term memory. Sepp hochreiter and jrgen schmidhuber. In *Neural Computation 9(8):17351780*, 1997.
- [61] Spacy. <https://github.com/free-variation/spaCy/tree/master/examples>. Accessed on 30 April 2019.
- [62] Marta Tatu and Dan Moldovan. A semantic approach to recognizing textual entailment. In *PASCAL*, 2005.
- [63] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems 30*, pages 5998–6008. Curran Associates, Inc., 2017. URL <http://papers.nips.cc/paper/7181-attention-is-all-you-need.pdf>.

- [64] Webopedia. <https://www.webopedia.com/TERM/N/NLP.html>. Accessed on 30 November 2018.
- [65] Adina Williams, Nikita Nangia, and Samuel Bowman. A broad-coverage challenge corpus for sentence understanding through inference. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 1112–1122. Association for Computational Linguistics, 2018. URL <http://aclweb.org/anthology/N18-1101>.
- [66] Ludwig Wittgenstein. In *Tractatus Logico-Philosophicus*, 1921.
- [67] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V. Le, and Mohammad Norouzi. Googles neural machine translation system: Bridging the gap between human and machine translation. In *arxiv.org*, 2016.
- [68] Peter Young, Alice Lai, Micah Hodosh, and Julia Hockenmaier. From image descriptions to visual denotations: New similarity metrics for semantic inference over event descriptions. In *Association for computer linguistics*, 2014.